

**CEFET/RJ - CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA
CELSO SUCKOW DA FONSECA**

**Match Movie: Um aplicativo facilitador de escolha
de filmes para grupos**

João Pedro de Souza Tents

Prof. Orientador:
Diego Borda Castro

**Rio de Janeiro
Fevereiro de 2025**

**CEFET/RJ - CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA
CELSO SUCKOW DA FONSECA**

Match Movie: Um aplicativo facilitador de escolha de filmes para grupos

João Pedro de Souza Tentis

Projeto final apresentado em cumprimento às
normas do Departamento de Educação
Superior do Centro Federal de Educação
Tecnológica Celso Suckow da Fonseca,
CEFET/RJ, como parte dos requisitos para
obtenção do título de Bacharel em Sistemas de
Informação.

Prof. Orientador:
Diego Borda Castro

**Rio de Janeiro
Fevereiro de 2025**

CEFET/RJ – Sistema de Bibliotecas / Biblioteca Uned Maria da Graça

T313m Tentis, João Pedro de Souza.

Match Movie : um aplicativo facilitador de escolha de filmes para grupos / João Pedro de Souza Tentis. — 2025.

xv, 70f.; fig. (color.) + apêndice : em PDF.

Projeto Final (Graduação) Centro Federal de Educação Tecnológica Celso Suckow da Fonseca, 2025.

Bibliografia: f. 66-70.

Orientador: Diego Cardoso Borda Castro

1. Software de aplicação - Desenvolvimento. 2. Filmes - Seleção. 3. React Native. I. Castro, Diego Cardoso Borda (orient.). II. Título.

CDD 005.1

Elaborada pela Bibliotecária Kellen Cristhiane Corrêa Faria CRB7/5496

JOÃO PEDRO DE SOUZA TENTIS

Match Movie: Um aplicativo facilitador de escolha de filmes para grupos

Trabalho de Conclusão de Curso,
apresentado ao Centro Federal de
Educação Tecnológica Celso Suckow da
Fonseca, como parte das exigências para
a obtenção do título de Bacharel em
Sistemas de Informação.

Rio de Janeiro, 24 de fevereiro de 2025.

BANCA EXAMINADORA

Documento assinado digitalmente
 **DIEGO CARDOSO BORDA CASTRO**
Data: 07/04/2025 19:05:34-0300
Verifique em <https://validar.iti.gov.br>

Prof. M.Sc. Diego Cardoso Borda Castro
Centro Federal de Educação Tecnológica (CEFET/RJ)

Documento assinado digitalmente
 **CRISTIANO FUSCHILLO**
Data: 08/04/2025 11:35:47-0300
Verifique em <https://validar.iti.gov.br>

Prof. M.Sc. Cristiano Fuschilo
Centro Federal de Educação Tecnológica (CEFET/RJ)

Documento assinado digitalmente
 **VITOR CARNEIRO MAIA**
Data: 07/04/2025 19:38:22-0300
Verifique em <https://validar.iti.gov.br>

D.Sc. Vitor Carneiro Maia
Lemobs

DEDICATÓRIA

Dedico este trabalho, primeiramente, aos meus pais, que me instruíram desde a infância que a autêntica felicidade vai além de qualquer justificativa. Embora não tenham tido a ocasião de finalizar seu ensino, empenharam-se ao máximo para que eu pudesse alcançar a minha formação. Com muita determinação, não apenas deslocam montanhas, mas também suportam o peso de um mundo inteiro para me sustentar.

São os meus principais modelos de resistência, afeição e renúncia. Para vocês, que se empenharam ao máximo para que eu alcançasse este ponto, dedico com amor sincero e gratidão cada página desta obra.

AGRADECIMENTOS

Agradeço aos meus amigos, pela parceria e apoio durante esta jornada. Aos meus professores, por compartilharem conhecimento e incentivarem meu crescimento. E, especialmente, ao meu orientador, pela orientação e dedicação que foram fundamentais para a conclusão deste trabalho.

Meu sincero obrigado a todos!

RESUMO

Este estudo centra-se na questão do paradoxo da escolha, que argumenta que uma grande diversidade de opções pode levar os indivíduos a enfrentar dificuldades em suas decisões. É adequado recordar o período das locadoras, nas quais os usuários dispunham de limitadas opções para escolher um filme, no entanto, se sentindo satisfeitos. Em contrapartida, a Netflix apresenta um catálogo repleto de diversos títulos, e muitas pessoas dedicam mais tempo à escolha de um filme do que efetivamente assistindo, por fim, o paradoxo expoe que a introdução da variável de grupos de pessoas pode aumentar o tempo consideravelmente. Diante disso, a aplicação Match Movie é apresentada tendo como objetivo auxiliar nesse problema. A aplicação integra diversas funcionalidades que tornam o processo mais dinâmico e eficiente, como a votação de filmes, catálogos atualizados e a recomendação de filmes com base em outros filmes. Este trabalho de conclusão de curso detalha todo o processo de desenvolvimento, desde a concepção inicial até a implementação final. São abordados os principais desafios encontrados, como a integração da API, a escolha da linguagem de programação, a criação da *UI* (Interface do usuário) / *UX* (Experiência do usuário) e a otimização do desempenho da aplicação, assim como as soluções adotadas para superá-los, demonstrando assim, todo o processo de de criação de uma aplicação.

Palavras-chaves: desenvolvimento; filmes; react-native; seleção; aplicativo; grupos; escolha

ABSTRACT

This research examines the paradox of choice, positing that an abundance of alternatives could confuse decision-making for individuals. It is vital to think back about the time period of video rental establishments, where customers had restricted selections for films, however experienced satisfaction. Conversely, Netflix provides a diverse inventory of movies, leading many individuals to invest more time selecting a film than watching it. Finally, the paradox demonstrates that including the variable of groups can significantly extend this period of time. Consequently, the Match Movie application is introduced to address this issue. The application has many features that enhance the process's dynamism and efficiency, including movie voting, updated catalogs, and recommendations based on other moveis. This last work defines the full development process, from the preliminary design to the final implementation. The challenges faced include API integration, selection of programming language, development of User Interface (UI) and User Experience (UX), and optimization of application performance, along with the solutions implemented that address these obstacles, thereby illustrating the comprehensive process of application development.

Keywords: development; movies; react-native; recommendation; app; groups; choice

Sumário

1	Introdução	1
1.1	Motivação e Contexto	1
1.2	Objetivo	2
1.3	Metodologia	3
1.4	Organização do texto	4
2	Fundamentação teórica	5
2.1	Processo de criação	5
2.2	Tecnologias.	6
2.2.1	Escolha da linguagem	7
2.2.2	React Native	9
2.2.3	Node.js	10
2.2.4	PostgreSQL	11
2.2.5	Prisma ORM	12
2.3	Paradoxo da Escolha	13
3	Trabalhos relacionados	15
3.1	Quais motivações são mencionadas para a criação de aplicativos para seleção de filmes em grupos?	20
3.2	Quais características os aplicativos encontrados utilizam para seleção de filmes em grupos?	21
4	Match Movie	23
4.1	Estratégia	23
4.2	Escopo	23
4.2.1	Funcionalidades e requisitos	23
4.2.2	Diagramação	25
4.3	Telas	26
4.3.1	Tela de Apresentação	26
4.3.2	Tela de Login e Registro	27
4.3.3	Telas de recuperação de senha	28

4.3.4	Tela de Catálogo	29
4.3.5	Tela de Perfil	30
4.3.6	Tela de Match - Sessões	30
4.3.7	Tela de Match - Histórico	31
4.3.8	Tela de Match - Votação	33
4.3.9	Tela de Match - Conclusão de votação	34
4.3.10	<i>Modals</i>	35
4.4	Marca	36
4.4.1	Paleta de cores	37
4.4.2	Logotipo	37
4.4.3	Escolha do nome	38
4.4.4	Tipografia	39
4.5	Desenvolvimento	40
4.5.1	Arquitetura da Aplicação	42
4.5.2	API	42
5	Avaliação e Resultados	48
5.1	Avaliação do TAM e Usabilidade	48
5.1.1	Facilidade de Uso Percebida (PEOU)	49
5.1.2	Utilidade Percebida (PU)	50
5.1.3	Intenção de Uso (IU)	50
5.2	Avaliação de UI/UX por meio de formulário	51
5.2.1	Facilidade de Uso	51
5.2.2	Design da Interface	52
5.2.3	Experiência do Usuário	53
5.2.4	Eficiência	53
5.2.5	Acessibilidade	54
5.2.6	Feedback do usuário	55
6	Considerações Finais e Trabalhos Futuros	56
	Apêndice	57
A	Diagrama de Classe	57
B	Diagrama de Caso de Uso	58

C	Diagrama de Sequência - Login de usuário	59
D	Diagrama de Sequência - Registro de usuário	59
E	Diagrama de Sequência - Obtenção de filmes	60
F	Diagrama de Sequência - Marcar filmes como favorito ou assistido	60
G	Diagrama de Sequência - Votação de filmes (match)	61
H	Arquitetura do sistema modelo C4	62
I	Swagger - Endpoints de autenticação	63
J	Swagger - Endpoints de usuários	63
K	Swagger - Endpoints de grupos	63
L	Swagger - Endpoints de filmes	64
M	Swagger - Endpoints de favoritos	64
N	Swagger - Endpoints de assistidos	64
O	Swagger - Endpoints de geolocalização e ingresso.com	65
P	Swagger - Endpoints de match (votação)	65
Referências		66

Lista de Figuras

FIGURA 1:	Metodologia utilizada e adaptada [??]	3
FIGURA 2:	Fluxo do processo de criação. Elaborado pelo autor.	5
FIGURA 3:	Protocolo de pesquisa aplicado. Elaborado pelo autor.	19
FIGURA 4:	Primeira tela ao abrir o aplicativo. Elaborado pelo autor.	26
FIGURA 5:	Telas de login e de registro. Elaboradas pelo autor.	27
FIGURA 6:	Telas de redefinição de senha. Elaboradas pelo autor.	28
FIGURA 7:	Tela de catálogo dos filmes e tela de informações. Elaboradas pelo autor.	29
FIGURA 8:	Tela de perfil do usuário. Elaborada pelo autor.	30
FIGURA 9:	Tela de match e tela de sessão. Elaboradas pelo autor.	31
FIGURA 10:	Tela para o usuário ver os históricos de suas sessões. Elaborada pelo autor.	32
FIGURA 11:	Tela de votação e pop-up de informação. Elaboradas pelo autor.	33
FIGURA 12:	Telas de conclusão de votação. Elaboradas pelo autor.	34
FIGURA 13:	<i>Modals</i> presentes nas telas de grupo. Elaboradas pelo autor.	35
FIGURA 14:	<i>Modal</i> para escolher filme de referência para grupo. Presente na tela da Figura 9.	36
FIGURA 15:	Paletas de cores primárias e secundárias. Fonte: <i>Pigment Shapefactory</i> .	37
FIGURA 16:	Logotipos com as cores usadas no aplicativo. Elaborado pelo autor.	38
FIGURA 17:	Exemplos das fontes Coiny e Inter. Fonte: <i>Google Fonts</i>	40
FIGURA 18:	Exemplo da fonte Coiny nas paletas de cores do aplicativo. Elaborado pelo autor.	41
FIGURA 19:	Diagrama de classe do Match Movie. Elaborado pelo autor.	57
FIGURA 20:	Diagrama de caso de uso do Match Movie. Elaborado pelo autor.	58
FIGURA 21:	Diagrama de Sequência - Login de usuário. Elaborado pelo autor.	59
FIGURA 22:	Diagrama de Sequência - Registro de usuário. Elaborado pelo autor.	59
FIGURA 23:	Diagrama de Sequência - Obtenção de filmes. Elaborado pelo autor.	60

FIGURA 24:	Diagrama de Sequência - Marcar filmes como favorito ou assistido. Elaborado pelo autor.	60
FIGURA 25:	Diagrama de Sequência - Votação de filmes (match). Elaborado pelo autor.	61
FIGURA 26:	Figura de exemplo da arquitetura do sistema no modelo C4. Elaborado pelo autor.	62
FIGURA 27:	Figura de exemplo dos endpoints de autenticação no Swagger. Elaborado pelo autor.	63
FIGURA 28:	Figura de exemplo dos endpoints de CRUD de usuários. Elaborado pelo autor.	63
FIGURA 29:	Figura de exemplo dos endpoints de CRUD de grupos. Elaborado pelo autor.	63
FIGURA 30:	Figura de exemplo dos endpoints de filmes. Elaborado pelo autor.	64
FIGURA 31:	Figura de exemplo dos endpoints de filmes favoritos do usuário. Elaborado pelo autor.	64
FIGURA 32:	Figura de exemplo dos endpoints de filmes assistidos do usuário. Elaborado pelo autor.	64
FIGURA 33:	Figura de exemplo dos endpoints de geolocalização e do ingresso.com. Elaborado pelo autor.	65
FIGURA 34:	Figura de exemplo dos endpoints votação de filmes. Elaborado pelo autor.	65

Lista de Tabelas

TABELA 1:	Comparação entre React Native e Java para desenvolvimento mobile	8
TABELA 2:	String de busca	16
TABELA 3:	Artigos selecionados para o estudo	18
TABELA 4:	Quantitativa de artigos em cada etapa da pesquisa.	19
TABELA 5:	Matriz de rastreabilidade	22
TABELA 6:	Endpoints utilizados para obtenção de informações de filmes. Fonte: <i>TheMovieDB.org</i>	45
TABELA 7:	Resultados da pesquisa sobre Facilidade de Uso Percebida (PEOU)	49
TABELA 8:	Resultados da pesquisa sobre Utilidade Percebida (PU)	50
TABELA 9:	Resultados da pesquisa sobre Intenção de Uso (IU)	51
TABELA 10:	Resultados da pesquisa sobre Facilidade de Uso	52
TABELA 11:	Resultados da pesquisa sobre Design da Interface	52
TABELA 12:	Resultados da pesquisa sobre Experiência do Usuário	53
TABELA 13:	Resultados da pesquisa sobre Eficiência	54
TABELA 14:	Resultados da pesquisa sobre Acessibilidade	54
TABELA 15:	Resultados da pesquisa sobre Feedback do Usuário	55

Acrônimos

Capítulo 1

Introdução

Este capítulo pretende apresentar o contexto, a motivação e os problemas abordados neste trabalho de conclusão de curso. Além disso, as questões de pesquisa, objetivos e a metodologia adotada para atingi-los também são apresentadas, bem como a organização do texto.

1.1 Motivação e Contexto

No mundo contemporâneo, a produção de entretenimento é uma atividade que cresce de maneira exponencial ao longo dos anos [???]. Entre os diversos tipos de entretenimento, destacam-se os jogos eletrônicos, vídeos de curta duração e, especialmente, as produções cinematográficas [?], sendo o foco deste trabalho. Um dos reflexos desse expressivo crescimento pode ser observado pelo alto valor econômico presente neste ambiente. "A Economia Criativa é um setor econômico em expansão global, estimulado pelo crescente interesse por produtos e serviços inovadores e criativos. De acordo com a Conferência das Nações Unidas sobre Comércio e Desenvolvimento (*UNCTAD*), a Economia Criativa movimentou US\$ 2,25 trilhões em 2015 e empregou cerca de 29,5 milhões de pessoas em todo o mundo"[?].

Essa abundância de opções, embora positiva, também apresenta um desafio significativo para o indivíduo: a dificuldade na escolha do que consumir. A imensa variedade disponível frequentemente resulta em um estado de indecisão e tempo ocioso, gerando desgaste e, por vezes, a desistência de consumir determinado conteúdo. Essa dificuldade de escolha é conhecida como "Paradoxo da escolha" que demonstra que, ao se ter muitas opções, é gerada uma indecisão e um aumento no tempo de escolha [?]. Vale lembrar que esse paradoxo pode possuir um agravante quando tratado de um grupo de pessoas, devido a razões como aumento da complexidade, indecisão coletiva, pressão em grupo e responsabilidade compartilhada, o que pode fazer esse tempo de decisão ser ainda maior [??].

De acordo com ?, que realizou um estudo em relação aos usuários da Netflix, um usuário leva em média 18 minutos para escolher o que assistir na plataforma, devido ao grande número de opções. Esse mesmo estudo mostra que, quando a questão é assistir a filme em conjunto,

nesse caso, em casal, 40% dos casais divergem no momento da decisão, demonstrando o agravante do paradoxo descrito acima. Por fim, o estudo demonstra que 70% dos usuários escolhem qual filme assistir apenas quando chegam no cinema, embora o número de títulos seja reduzido, ao levar em consideração um grupo de pessoas, esse tempo de escolha pode ser desgastante [?].

Diante desse cenário e dos problemas descritos, surge a questão de pesquisa deste trabalho: como otimizar a escolha de filmes em grupos diante da abundância de opções. Visando auxiliar na resposta desta pergunta, este trabalho tem como principal objetivo o desenvolvimento de uma aplicação mobile destinada a simplificar a escolha de filmes para grupos de usuários. A importância de tal ferramenta se justifica pela necessidade atual de otimizar o tempo e reduzir o esforço despendido em uma tarefa aparentemente "simples", mas que se tornou complexa devido à vasta quantidade de opções disponíveis. Estudos indicam que o excesso de escolhas pode levar à paralisia decisória e insatisfação [?]. Além disso, o crescimento significativo dos serviços de streaming ampliou ainda mais as opções: entre maio de 2021 e maio de 2022, o uso de streaming aumentou mais de 21%, representando quase 32% do tempo total de TV [?]. Dessa forma, espera-se que, com uma ferramenta, seja possível economizar tempo e reduzir o desgaste.

Além do objetivo central de responder à questão de pesquisa, a aplicação proposta inclui várias funcionalidades específicas que visam aprimorar a experiência do usuário, nas quais se destacam: um catálogo de filmes atualizado em tempo real, seleção de filmes com base em filtros e sugestões de filmes similares, criação de grupos para votações e redirecionamento para a compra de ingressos do filme escolhido. Essas funcionalidades visam ajudar a otimizar a maneira como os grupos escolhem e consomem produções cinematográficas, tornando o processo mais dinâmico, eficiente e agradável.

1.2 Objetivo

Na tentativa de otimizar o processo de seleção de filmes em conjunto, o presente trabalho propõe o desenvolvimento de uma ferramenta para seleção de filmes em grupos, visando otimizar este processo. Este objetivo geral pode ser decomposto em objetivos específicos:

- Caracterizar o estado da arte sobre aplicações com o objetivo de selecionar filmes em grupo: Realizar uma comparação crítica entre as ideias do aplicativo proposto e outros aplicativos encontrados na literatura, como o Queue e o Filmy, focando em aspectos como

eficiência, simplicidade, usabilidade e satisfação do usuário;

- Avaliar a experiência do usuário: examinar como a abordagem simplificada do Match Movie influencia a experiência dos usuários na seleção de filmes, utilizando métricas de eficiência e satisfação;
- Implementar um aplicativo para facilitar a seleção de filmes em grupos;
- Propor Melhorias Futuras: Oferecer recomendações para futuras melhorias no Match Movie, baseando-se nas análises e comparações realizadas, visando aprimorar ainda mais a experiência do usuário e a eficácia da aplicação.

1.3 Metodologia

Este trabalho seguiu uma metodologia inspirada nos modelos de *Action Design Research* [?] e *Design Science Research* [?] que foram adaptados para o contexto do trabalho, sendo feitas algumas simplificações.

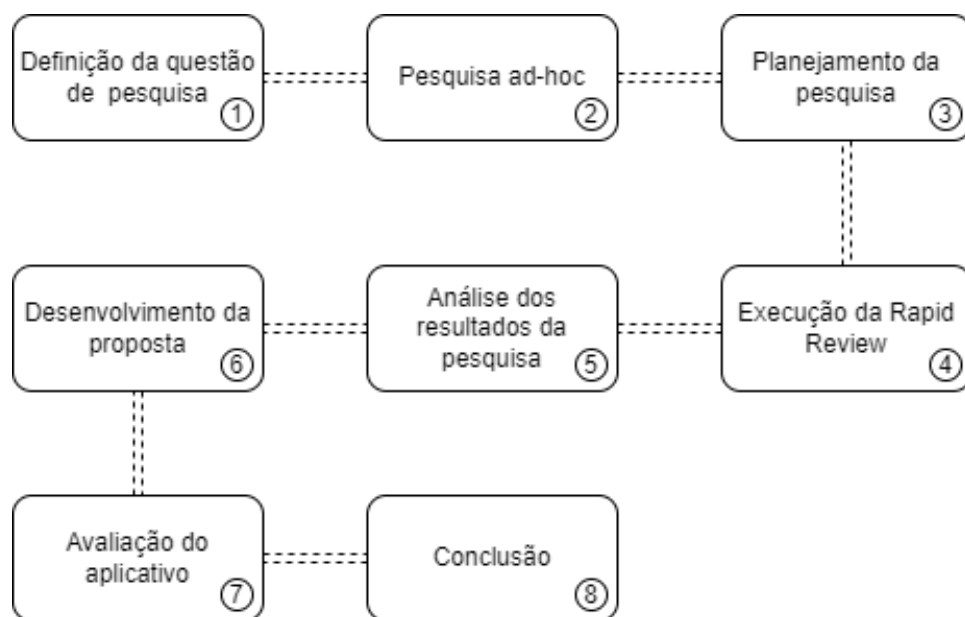


Figura 1: Metodologia utilizada e adaptada [??]

Cada uma das atividades apresentadas na Figura 1 é descrita com mais detalhes a seguir:

- **Definição da questão de pesquisa:** definição do problema que o trabalho se propõe a tentar resolver;

- **Pesquisa ad-hoc:** Elaborar um estudo inicial através de pesquisas no Google Scholar e diretamente na internet para ter um primeiro contato com a área pesquisada;
- **Planejamento da pesquisa:** Definição do protocolo de pesquisa e motor de busca que seria utilizado para a realização do estudo;
- **Análise dos resultados da pesquisa:** analisar os artigos encontrados e extrair as informações definidas no protocolo de pesquisa;
- **Desenvolvimento da proposta:** Desenvolvimento do aplicativo para seleção de filmes em grupos;
- **Avaliação do aplicativo:** Avaliação do aplicativo desenvolvido;
- **Conclusão:** Avaliar as informações coletadas na avaliação visando melhorar o aplicativo desenvolvido.

1.4 Organização do texto

Este trabalho de conclusão de curso está organizado em cinco capítulos. Neste capítulo, foi apresentado o contexto, bem como a motivação e a questão de pesquisa.

O Capítulo 2 apresenta a fundamentação teórica sobre alguns assuntos que ajudam no entendimento dos demais capítulos.

O Capítulo 3 apresenta uma breve discussão sobre os resultados encontrados em uma *Rapid Review* [?] sobre o tema de aplicativos para seleção de filmes em grupos, demonstrando as características de cada um que foi encontrado.

O Capítulo 4 apresenta a proposta de solução do problema apresentado neste trabalho, no qual foi proposto um aplicativo para seleção de filmes em grupo, visando otimizar tal processo; nele, suas características, diagramações, tecnologias utilizadas e avaliações são descritas em mais detalhes.

O Capítulo 5 apresenta uma discussão final sobre o conteúdo que foi apresentado nesse trabalho de pesquisa, suas contribuições, limitações e trabalhos futuros.

Capítulo 2

Fundamentação teórica

2.1 Processo de criação

No processo de criação do Match Movie, foi seguido um fluxo (Figura 2) estruturado e inspirado em processos de design de UX amplamente utilizados por empresas líderes no setor, conforme descrito por ?. Conforme descrito na literatura, o design de UX envolve diversas etapas interdependentes, começando pela pesquisa e descoberta, na qual buscamos compreender profundamente o público-alvo e suas necessidades. Essa fase inicial envolveu pesquisas de mercado, análise de concorrentes e entrevistas com potenciais usuários para garantir que o produto atenderia às expectativas.

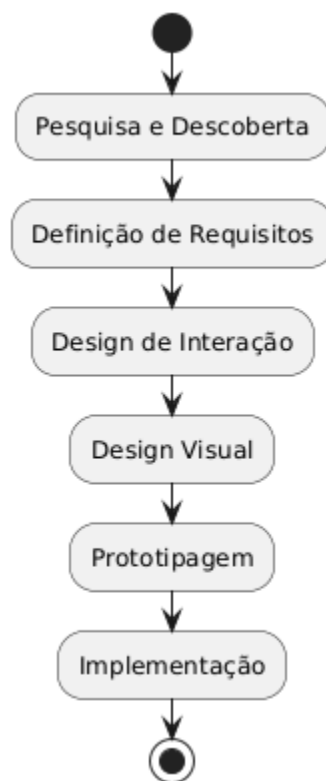


Figura 2: Fluxo do processo de criação. Elaborado pelo autor.

Após a fase de descoberta, a definição de requisitos é realizada. Nessa etapa, as funcionali-

dades e características essenciais que o sistema deveria possuir são estabelecidas, alinhando-as às necessidades dos usuários e aos objetivos estratégicos do projeto. Em seguida, o design de interação é realizado, no qual a estrutura do produto é definida, criando fluxos de navegação intuitivos e uma arquitetura de informações clara e eficiente.

A etapa de design visual foi fundamental para conferir personalidade ao Match Movie. A paleta de cores, tipografia e elementos gráficos que representassem a identidade da marca de forma consistente e atraente foram selecionados cuidadosamente. Esses aspectos foram definidos para criar uma conexão emocional com o usuário e transmitir os valores centrais do produto. Após consolidar o design visual, foram realizados testes de usabilidade para validar as escolhas. No teste de navegação com cenários, os usuários receberam tarefas como encontrar recomendações de filmes baseadas em seus gostos. Os resultados mostraram que a maioria conseguiu navegar intuitivamente, mas ajustes na hierarquia visual foram necessários para destacar melhor os filtros de recomendação. No teste de tempo de conclusão de tarefa, foi medido a rapidez com que os usuários criavam um perfil e selecionavam seus filmes favoritos. Foi identificado que, em média, a conclusão levava mais tempo do que o esperado, levando à simplificação da forma de marcar filme como favorito. Esse processo iterativo permitiu identificar ajustes necessários antes da implementação final, garantindo uma experiência de usuário mais fluida e eficiente.

Por fim, a implementação marcou o desenvolvimento do produto completo, com melhorias contínuas baseadas no feedback dos usuários. Detalhes mais específicos sobre a construção da marca, escolha da paleta de cores, elementos de UI/UX e tipografia estão detalhados no capítulo do Match Movie. Entretanto, este resumo serve para fundamentar as escolhas feitas no processo de criação do Match Movie, evidenciando o compromisso com uma experiência de usuário sólida e bem fundamentada.

Para embasar o processo, referimo-nos ao estudo de [?], que discute a pesquisa acadêmica sobre o designer de UX e oferece uma revisão sistemática da literatura.

2.2 Tecnologias.

O desenvolvimento do aplicativo teve como escolha de sua linguagem primária, inicialmente, Java, porém, após estudos e análises, foi decidido migrar para outra tecnologia, discutida nos tópicos abaixo. Para um melhor aproveitamento e um desenvolvimento coeso, foram escolhidas as seguintes tecnologias para desenvolver a aplicação: React Native para o aplicativo, Node.js com Nest.js para a API, Prisma ORM para abstração e facilitação do uso do

banco de dados e PostgreSQL como banco de dados. Os principais fatores de decisão foram: conhecimento do autor nas linguagens, facilidade de integração, coerência no ecossistema e comunicação fluida entre as camadas do aplicativo.

2.2.1 Escolha da linguagem

Inicialmente, no desenvolvimento do aplicativo, foram consideradas as seguintes tecnologias: Java, React Native, Flutter e Kotlin. Kotlin foi rapidamente descartado devido à falta de conhecimento do autor na linguagem, o que tornaria a curva de aprendizado significativamente mais longa para o projeto. Da mesma forma, Flutter, desenvolvido com base na linguagem Dart, também apresentou desafios semelhantes. Embora seja uma tecnologia promissora, Dart é menos comum entre desenvolvedores, aumentando a complexidade de aprendizado, como observado por ?.

Java foi, então, a escolha inicial para o desenvolvimento. Essa decisão foi motivada pela busca por desempenho otimizado, considerando que linguagens nativas e menor tempo de execução, o que é essencial para aplicações com demandas de alta *performance*, conforme indicado por ?. A escolha também foi fortalecida pela vasta documentação e suporte da comunidade, tornando a implementação inicial mais fluida.

Após alguns meses de desenvolvimento e o lançamento da versão 1 do aplicativo, uma auditoria de arquitetura e escalabilidade foi realizada para avaliar o desempenho e a eficiência geral do projeto. Os resultados indicaram que, embora Java tenha atendido às expectativas de *performance*, a separação entre tecnologias no *front-end* e *back-end* estava gerando desafios em termos de manutenção e escalabilidade. A auditoria revelou que a estrutura baseada em Java no *back-end* e uma tecnologia distinta no *front-end* aumentavam a complexidade do desenvolvimento, exigindo um bom domínio para cada tecnologia, domínio esse que o autor não possui com Java, o que dificultava a manutenção e tornava o processo de correção de bugs e implementação de novas funcionalidades mais lento. Além disso, a necessidade de comunicação entre serviços escritos em linguagens diferentes exigia integrações mais complexas, aumentando o risco de incompatibilidades e impactando a escalabilidade do sistema. Como a *Application Programming Interface: Interface de Programação de Aplicação (API)* foi desenvolvida em Node.js, um *framework* robusto e amplamente adotado para aplicações *back-end* atuais, de acordo com ?.

Tabela 1: Comparação entre React Native e Java para desenvolvimento mobile

Critério	React Native	Java
Natureza	Framework multiplataforma	Linguagem nativa
Facilidade de Uso	Alta, devido à reutilização de código	Moderada, exige conhecimento específico
Performance	Boa, mas inferior às linguagens nativas	Excelente para requisitos críticos
Curva de Aprendizado	Rápida para quem já conhece JavaScript	Íngreme para iniciantes
Reutilização de Código	Alta (compartilhamento entre iOS e Android)	Baixa, código específico por plataforma
Comunidade	Grande, com suporte extensivo de bibliotecas	Grande, mas mais focada em Android
Atualizações	Constantes pela comunidade open-source	Estáveis pelo Android SDK
Acesso ao Hardware	APIs externas e módulos de suporte	Acesso direto e otimizado
Desempenho Gráfico	Boa para UI responsiva	Boa para gráficos complexos
Ecossistema	Integração com Node.js e APIs web	Baseado no Android SDK
Casos de Uso	Multiplataforma e interfaces médias	Nativo, ideal para alto desempenho

Neste contexto, React Native surgiu como a solução mais adequada. Sendo um *framework* baseado em JavaScript, assim como Node.js, ele permite uma integração fluida entre o *front-end* e o *back-end*, otimizando o fluxo de desenvolvimento e a manutenção do código. Além disso, React Native oferece a vantagem de desenvolvimento multiplataforma, permitindo que uma única base de código seja usada para iOS e Android, reduzindo esforços e custos de manutenção. A escolha foi reforçada pela curva de aprendizado mais suave, dado que o autor já possuía experiência com JavaScript, e pela capacidade do React Native de criar interfaces de usuário responsivas e eficientes, mesmo com desempenho um pouco inferior ao das linguagens nativas. Mais detalhes podem ser observados na Tabela 1.

Essa mudança representou um avanço significativo para o projeto, permitindo maior consistência no desenvolvimento, melhor alinhamento tecnológico entre as camadas da aplicação e um processo de manutenção mais eficiente. A decisão de transitar para React Native não apenas otimizou o desenvolvimento, mas também alinhou a arquitetura do aplicativo às melhores práticas do mercado, consolidando o uso de tecnologias modernas e amplamente suportadas [?].

2.2.2 React Native

Em 2015, na conferência de React.js, o Facebook apresentou o React Native, um *framework* inovador para o desenvolvimento de aplicativos móveis baseados em React.js. O React Native é *open-source* e foi projetado para simplificar o desenvolvimento multiplataforma, permitindo que desenvolvedores criem aplicativos nativos para iOS e Android utilizando a mesma base de código em JavaScript. Segundo [?], sua principal vantagem está na redução de esforços de aprendizado e desenvolvimento, já que elimina a necessidade de dominar linguagens nativas distintas, como Swift para iOS e Kotlin para Android.

Uma das características fundamentais do React Native é a capacidade de traduzir componentes de interface de usuário escritos em JavaScript para componentes nativos específicos de cada plataforma. Isso garante que o aplicativo mantenha o desempenho e a aparência de um aplicativo nativo, mesmo sendo desenvolvido em um *framework* multiplataforma. Essa abordagem híbrida supera as limitações de tecnologias baseadas apenas em *WebView*, como Ionic, e oferece maior flexibilidade para renderizar gráficos e componentes de maneira eficiente em ambas as plataformas [?].

Além disso, o React Native se destaca por sua arquitetura modular, que permite integrar facilmente pacotes e bibliotecas de terceiros, ampliando sua funcionalidade. Ferramentas populares, como Context API para gerenciamento de estado e React Navigation para navegação, são amplamente suportadas, o que facilita a criação de aplicativos robustos e escaláveis [?]. Outro ponto positivo é o suporte à atualização *over-the-air* (OTA), um método de distribuição de atualizações de software de forma remota, sem a necessidade de reinstalar ou baixar manualmente uma nova versão do aplicativo pelas lojas oficiais (Google Play e App Store), por meio de serviços como *CodePush*, que permite a entrega de atualizações sem a necessidade de passar pelos processos tradicionais de publicação em lojas de aplicativos.

Porém, é importante mencionar que o React Native apresenta algumas limitações, como o desempenho ligeiramente inferior em comparação com aplicativos desenvolvidos nativamente, especialmente em cenários que envolvem gráficos intensivos ou cálculos complexos [?]. Ainda assim, para aplicativos com interfaces de usuário responsivas e interações moderadas, ele oferece uma solução altamente eficiente.

De acordo com [?], o React Native também se beneficia de uma comunidade de desenvolvedores ativa e em constante crescimento, o que garante a evolução contínua do *framework*, bem

como a disponibilidade de recursos educacionais e suporte técnico. Esse ecossistema robusto reduz as barreiras para novos desenvolvedores e permite que equipes adotem rapidamente as melhores práticas no desenvolvimento multiplataforma.

No contexto do Match Movie, uma decisão importante no desenvolvimento deste projeto foi a utilização do **Expo** em vez do *React Native CLI* como ferramenta principal. O **Expo** é um conjunto de ferramentas e serviços que simplifica o desenvolvimento de aplicativos em React Native, eliminando a necessidade de configuração manual de ambientes nativos, como *Xcode* para *iOS* e *Android Studio* para *Android*. Enquanto o *React Native CLI* oferece maior controle sobre a personalização do código nativo, ele exige um conhecimento mais aprofundado das plataformas subjacentes e pode demandar um tempo significativo para configurar corretamente bibliotecas nativas. O Expo, por outro lado, proporciona uma experiência mais ágil e acessível, especialmente útil para equipes menores ou para projetos que não exigem modificações extensivas em código nativo, como afirmado no estudo de ?. Além disso, o Expo facilita ainda mais a implementação de atualizações via *over-the-air (OTA)* e a depuração do aplicativo com recursos como o *Expo Go*, que permite testar rapidamente mudanças sem precisar recompilar o código. No entanto, o uso do Expo pode apresentar algumas limitações, como a dependência de pacotes pré-compilados e o suporte restrito a certas bibliotecas nativas mais avançadas. Ainda assim, considerando a necessidade de rapidez no desenvolvimento e a simplicidade na manutenção, o Expo foi escolhido como a solução mais adequada para este projeto. [?]

No geral, a escolha do React Native para este projeto foi motivada por sua capacidade de unificar o *front-end* e o *back-end*, ambos baseados em JavaScript, o que simplifica o desenvolvimento, reduz os custos de manutenção e melhora a eficiência geral da equipe. Essa integração se mostrou vantajosa ao alinhar o projeto às demandas atuais do mercado por soluções rápidas e eficazes.

2.2.3 Node.js

O Node.js é uma plataforma de desenvolvimento que permite a execução de código JavaScript no lado do servidor, utilizando o motor V8 do Google. Criado por Ryan Dahl em 2009, o Node.js revolucionou o desenvolvimento de aplicações ao introduzir um modelo assíncrono e orientado a eventos, permitindo que servidores lidassem com operações de entrada e saída (I/O) de maneira eficiente e sem bloqueios.[?]. Essa abordagem tornou o Node.js uma escolha popular para sistemas que exigem alta escalabilidade e respostas em tempo real, como aplicativos de

mensagens, plataformas de streaming e sistemas de votação.

Um dos elementos mais inovadores do Node.js é seu modelo de event loop, que utiliza uma única thread para gerenciar milhares de conexões simultaneamente. Diferentemente de arquiteturas tradicionais que criam uma nova thread para cada solicitação, o Node.js mantém os recursos leves e eficientes, mesmo sob alta carga. Esse modelo não apenas melhora o desempenho, mas também reduz significativamente o custo de operação do servidor, especialmente em sistemas que dependem de respostas rápidas e consistentes [?].

Outro ponto de destaque do Node.js é sua comunidade ativa e expansiva, que contribui para um ecossistema robusto por meio do *Node Package Manager (npm)*, gerenciador de pacotes do Node.js, utilizado para instalar, atualizar e gerenciar bibliotecas e dependências em projetos JavaScript. Ele permite que desenvolvedores integrem pacotes de terceiros de forma eficiente, facilitando o desenvolvimento e a manutenção de aplicações. Com milhares de pacotes disponíveis, o Node.js oferece ferramentas modulares que aceleram o desenvolvimento e facilitam a integração de funcionalidades complexas. Além disso, o uso de JavaScript tanto no *front-end* quanto no *back-end* simplifica o fluxo de trabalho da equipe de desenvolvimento, promovendo maior consistência no código e facilitando a manutenção [?].

No contexto do aplicativo Match Movie, o Node.js foi escolhido para o desenvolvimento da API por sua eficiência em lidar com múltiplas requisições simultâneas, característica fundamental para o sistema de votação de filmes em grupo. O modelo de *event loop* assegura que o aplicativo funcione de maneira fluida, mesmo quando vários usuários interagem ao mesmo tempo, votando em diferentes filmes. Além disso, a modularidade do Node.js, associada à vasta gama de pacotes disponíveis no npm, facilitou a integração com o *front-end*, desenvolvido em React Native, criando um ecossistema coeso e eficiente. Essa combinação permite ao Match Movie oferecer uma experiência de usuário fluida e altamente responsiva, alinhada às expectativas de seu público.

2.2.4 PostgreSQL

O PostgreSQL é um sistema de gerenciamento de banco de dados objeto-relacional de código aberto, reconhecido por sua robustez, extensibilidade e conformidade com os padrões SQL. Desenvolvido inicialmente na Universidade da Califórnia, Berkeley, o PostgreSQL evoluiu para suportar uma ampla variedade de tipos de dados e funcionalidades avançadas, tornando-se uma escolha popular para aplicações que exigem integridade e consistência dos dados [?].

Vantagens do PostgreSQL:

- *Suporte a Tipos de Dados Avançados:* O PostgreSQL oferece suporte a uma variedade de tipos de dados, incluindo JSON, XML e arrays, proporcionando flexibilidade no armazenamento e manipulação de dados complexos.
- *Conformidade com ACID:* Garante transações atômicas, consistentes, isoladas e duráveis, essenciais para aplicações que requerem alta confiabilidade e integridade dos dados.
- *Extensibilidade:* Permite a criação de funções, tipos de dados e operadores personalizados, adaptando-se às necessidades específicas de diferentes aplicações [?].

Para o Match Movie, o PostgreSQL foi selecionado como banco de dados principal devido à sua capacidade de lidar com estruturas de dados complexas e à necessidade de manter a integridade das transações durante as votações de filmes pelos usuários. A conformidade com ACID assegura que cada voto seja registrado de forma consistente e confiável, enquanto o suporte a tipos de dados avançados permite a implementação de funcionalidades adicionais, como armazenamento de preferências dos usuários e histórico de interações. A combinação do PostgreSQL com o Prisma ORM e o Node.js resultou em uma arquitetura sólida e eficiente, capaz de suportar o crescimento e a complexidade do aplicativo.

2.2.5 Prisma ORM

O Prisma ORM é uma ferramenta moderna de mapeamento objeto-relacional que facilita a interação entre aplicações Node.js e bancos de dados. Diferentemente de ORMs tradicionais, o Prisma oferece uma abordagem centrada no desenvolvedor, proporcionando uma experiência mais intuitiva e eficiente na manipulação de dados. Ele suporta diversos bancos de dados, incluindo PostgreSQL, MySQL, SQLite e SQL Server, permitindo flexibilidade na escolha do sistema de gerenciamento de banco de dados [?].

Vantagens do Prisma ORM:

- *Abstração Simplificada:* O Prisma abstrai a complexidade das consultas SQL, permitindo que os desenvolvedores interajam com o banco de dados utilizando uma sintaxe mais próxima da linguagem de programação, o que acelera o desenvolvimento e reduz erros [?].

- *Segurança e Consistência*: Com suporte a tipagem estática, o Prisma ajuda a evitar erros comuns em tempo de execução, garantindo maior consistência e segurança no acesso aos dados [?].
- *Migrations Automatizadas*: O Prisma Migrate facilita a gestão de alterações no esquema do banco de dados, permitindo que migrações sejam realizadas de forma segura e controlada [?].

No contexto do Match Movie, o Prisma ORM foi escolhido para intermediar a comunicação entre a API, construída em Node.js, e o banco de dados PostgreSQL. Essa decisão permitiu uma integração mais fluida e eficiente, aproveitando as vantagens de tipagem estática e migrações automatizadas oferecidas pelo Prisma. A capacidade do Prisma de simplificar operações complexas no banco de dados contribuiu para um desenvolvimento mais ágil e um código-base mais limpo, facilitando a manutenção e evolução do aplicativo.

2.3 Paradoxo da Escolha

De acordo com ? em seu livro *O Paradoxo da Escolha*, a crença de que um maior número de opções resulta em mais benefícios ao usuário é, na realidade, um equívoco. Em vez de facilitar a decisão, o excesso de escolhas gera indecisão e insatisfação, levando as pessoas a buscar constantemente mais alternativas, mesmo quando isso dificulta a tomada de decisão. Essa perspectiva é especialmente relevante no contexto do Match Movie, que visa facilitar a escolha de filmes para grupos, onde a vasta gama de opções disponíveis pode dificultar a tomada de decisão.

Pesquisas subsequentes corroboram as observações de ?. Um estudo ? investigou o impacto do número de opções na satisfação do consumidor, diferenciando entre "*maximizadores*" (aqueles que buscam a melhor opção possível) e "*satisficers*" (aqueles que se contentam com uma opção que seja boa o suficiente). Os resultados indicaram que um número excessivo de opções pode diminuir a satisfação, especialmente entre os maximizadores, que tendem a experimentar maior arrependimento e dúvida após a escolha.

No contexto de um aplicativo que auxilia grupos na seleção de filmes, é crucial considerar o Paradoxo da Escolha. Ao simplificar o processo de decisão e possivelmente limitar o número de opções apresentadas, o aplicativo pode reduzir a sobrecarga cognitiva e aumentar a satisfação dos usuários com suas escolhas. Implementar filtros personalizados ou recomendações cura-

das pode ajudar a direcionar os usuários para opções que atendam às preferências coletivas do grupo, tornando o processo mais eficiente e agradável.

Ao abordar o Paradoxo da Escolha, o aplicativo não apenas facilita a tomada de decisão, mas também melhora a experiência do usuário, minimizando a ansiedade associada à seleção entre inúmeras opções. Essa abordagem, alinhada com as descobertas de Schwartz e pesquisas subsequentes, pode levar a uma maior satisfação e engajamento dos usuários.

Capítulo 3

Trabalhos relacionados

Embora este trabalho seja um *TCC* com um viés mais prático, o presente trabalho não abdicou de seguir um protocolo de pesquisa bem definido e reproduzível para encontrar seus trabalhos relacionados. No entanto, não seguiu métodos exaustivos como mapeamentos sistemáticos ou revisões da literatura, preferindo seguir um método de pesquisa com um retorno de resultados mais rápidos, optando pela execução de uma *Rapid Review* [?].

O processo de pesquisa apresentado nesse capítulo foi conduzido durante o período de maio de 2024 a julho de 2024 e visou identificar os trabalhos que já haviam sido realizados na área de aplicativos para seleção de filmes em grupos, buscando responder duas perguntas principais que estavam diretamente ligadas ao tema de pesquisa, sendo elas: (1) Quais motivações são mencionadas para a criação de aplicativos para seleção de filmes em grupos? (2) Quais características os aplicativos encontrados utilizam para a seleção de filmes em grupos?

Para definição da *string* de busca foi utilizado o *framework PICOC* (*population, intervention, comparison, outcome e context*) [?], onde quatro das cinco partes foram utilizadas visando auxiliar na fragmentação da *string* visando uma maior efetivação dos resultados encontrados. As palavras que formam a *string* de busca foram definidas em inglês buscando encontrar mais resultados sobre o tema. Além disso, algumas delas foram colocadas com o caractere “*” no final, para que fosse possível trazer qualquer termo que seja a derivação da palavra original.

Mediante os sinônimos identificados, foi possível derivar as *strings* de busca para o trabalho. A *string* foi definida agrupando as palavras-chave do mesmo domínio com o operador lógico “OR” e agrupando-se os domínios com o operador lógico “AND”. Cada uma das partes do *PICOC* e a **string** de busca utilizada podem ser visualizadas na Tabela 2.

Como foi mencionado no início do capítulo, esse trabalho buscou utilizar métodos de busca que retornassem um resultado mais rápido com um menor esforço, mas não abrindo mão da utilização de um método sistemático (ou seja, baseado em um processo ou protocolo estruturado e repetitivo). Devido a essas razões, optou-se pela execução de uma *Rapid Review* [?] utilizando uma única base de pesquisa, sendo escolhida a **Scopus** (<https://www.scopus.com>). A decisão foi motivada pelo seu extenso repositório de artigos científicos revisados por pares,

abrangendo diversas áreas do conhecimento e garantindo um alto nível de credibilidade das fontes. Além disso, a Scopus oferece uma estrutura avançada de busca, permitindo a criação de *strings* de pesquisa mais precisas, com suporte ao uso de caracteres curinga ('*') para capturar variações de palavras e sem restrições quanto ao número de operadores booleanos (AND e OR). Esses fatores tornam a base ideal para a presente revisão, garantindo um levantamento bibliográfico abrangente e relevante para os objetivos do estudo.

P	<i>movie*, film*</i>
I	<i>recommendation*, select*, suggestion*, decision*</i>
C	<i>Não se aplica</i>
O	<i>software*, application*, website*</i>
C	<i>group*, session*, collaborat*, together</i>
TITLE-ABS-KEY (("movie"OR "movies"OR "film"OR "films") AND ("recommendation*"OR "select*"OR "suggestion*"OR "decision*") AND ("software*"OR "application*"OR "website*") AND ("group*"OR "collaborative"OR "together"OR "session*")) AND PUBYEAR > 2014 AND PUBYEAR < 2025 AND (LIMIT-TO (SUBJAREA , "COMP"))	

Tabela 2: String de busca

Após a coleta inicial, os artigos foram filtrados com base em critérios de inclusão e exclusão pré-definidos de acordo com seu título, resumo e leitura completa, utilizando a plataforma do Parsifal (<https://parsif.al>) para organização da pesquisa, focando em estudos que abordassem diretamente os aspectos de desenvolvimento, usabilidade e comparação de aplicativos de escolha de filmes. Foram selecionados artigos que fornecessem dados empíricos e análises relevantes ao objetivo do estudo.

Cabe destacar que, na formulação da *string* de busca utilizada na base **Scopus**, não foram incluídos termos relacionados ao **Paradoxo da Escolha**. Essa decisão se deve ao fato de que a concepção do paradoxo como parte da argumentação do estudo foi realizada após a execução da busca inicial, o que impossibilitou sua consideração na definição dos critérios de pesquisa. Além disso, acredita-se que a inclusão de palavras-chave diretamente relacionadas ao conceito do paradoxo não agregaria significativamente à revisão sistemática, visto que se trata de um tema altamente nichado na literatura acadêmica. Assim, a abordagem sobre o Paradoxo da Escolha foi embasada em referências já estabelecidas na literatura e sua aplicação contextualizada com os objetivos do estudo, garantindo uma argumentação sólida sem comprometer a coerência da pesquisa bibliográfica inicial.

Critério de inclusão

1. O artigo deve estar no contexto de aplicações para seleção de filmes em grupos e/ou recomendações de filmes;
2. O artigo deve ser um estudo primário ou secundário;
3. O artigo deve responder pelo menos uma pergunta das questões de pesquisa.

Critério de exclusão

1. Capítulos de livro e chamada para congressos devem ser excluídos;
2. Artigos que não puderem ser acessados por completado devem ser excluídos;
3. Estudos que não estejam dentro da área de Ciência da Computação ou Engenharia devem ser excluídos.

Após a leitura por título, resumo e leitura completa seguindo os critérios listados acima, os dados dos artigos selecionados foram sintetizados para identificar padrões e *insights* relevantes. Foi dada atenção especial às métricas de eficiência, simplicidade e satisfação do usuário. Por fim, os resultados foram interpretados à luz das perguntas de pesquisa, discutindo como a simplicidade e o foco da aplicação proposta podem influenciar positivamente a experiência do usuário em comparação com aplicativos encontrados na pesquisa. A seguir, os resultados encontrados para cada uma das perguntas de pesquisa são discutidos em mais detalhes. O detalhamento do protocolo de pesquisa é demonstrado na Figura 3, após isso, são aplicados os critérios de inclusão e exclusão, como mencionados acima. Além disso, todos os artigos selecionados após a aplicação de todos os critérios da revisão podem ser vistos na Tabela 3. Por fim, todo o processo de execução com o quantitativo dos artigos em cada etapa pode ser visualizado na Tabela 4, sendo complementado pela figura 3, que demonstra todo o processo seguido de forma visual. Vale lembrar que o processo de criação da *string* de busca é um processo iterativo, sendo representado por um símbolo de *loop*, ocorrendo por meio da criação de uma string e calibração da mesma, tentando ser o mais assertivo possível no retorno dos artigos que estejam dentro do contexto do trabalho.

	Nome	Citação	Ano
1	SQUIRREL 2.0: Fairness Explanations for Sequential Group Recommendations	[?]	2024
2	Fair Sequential Group Recommendations in SQUIRREL Movies	[?]	2023
3	Movie Recommendation using Collaborative Filtering to Make Accurate Predictions	[?]	2022
4	User Profile-Based Recommendation Engine Mitigating the Cold-Start Problem	[?]	2022
5	Improving Graph-Based Movie Recommender System Using Cinematic Experience	[?]	2022
6	Sequence recommendations for groups: A dynamic approach to balance preferences	[?]	2022
7	A comprehensive analysis on movie recommendation system employing collaborative filtering	[?]	2021
8	FindMoviez: A Movie Recommendation System	[?]	2020
9	Movie Recommendation System	[?]	2019
10	Learning with linear mixed model for group recommendation systems	[?]	2019
11	Item-Based Collaborative Filtering in Movie Recommendation in Real time	[?]	2018
12	A user rating based collaborative filtering approach to predict movie preferences	[?]	2017
13	An algorithm for movie classification and recommendation using genre correlation	[?]	2016
14	A systematic literature review on the state of research and practice of collaborative filtering technique and implicit feedback	[?]	2016
15	Movie Recommender system using the user's psychological profile	[?]	2015

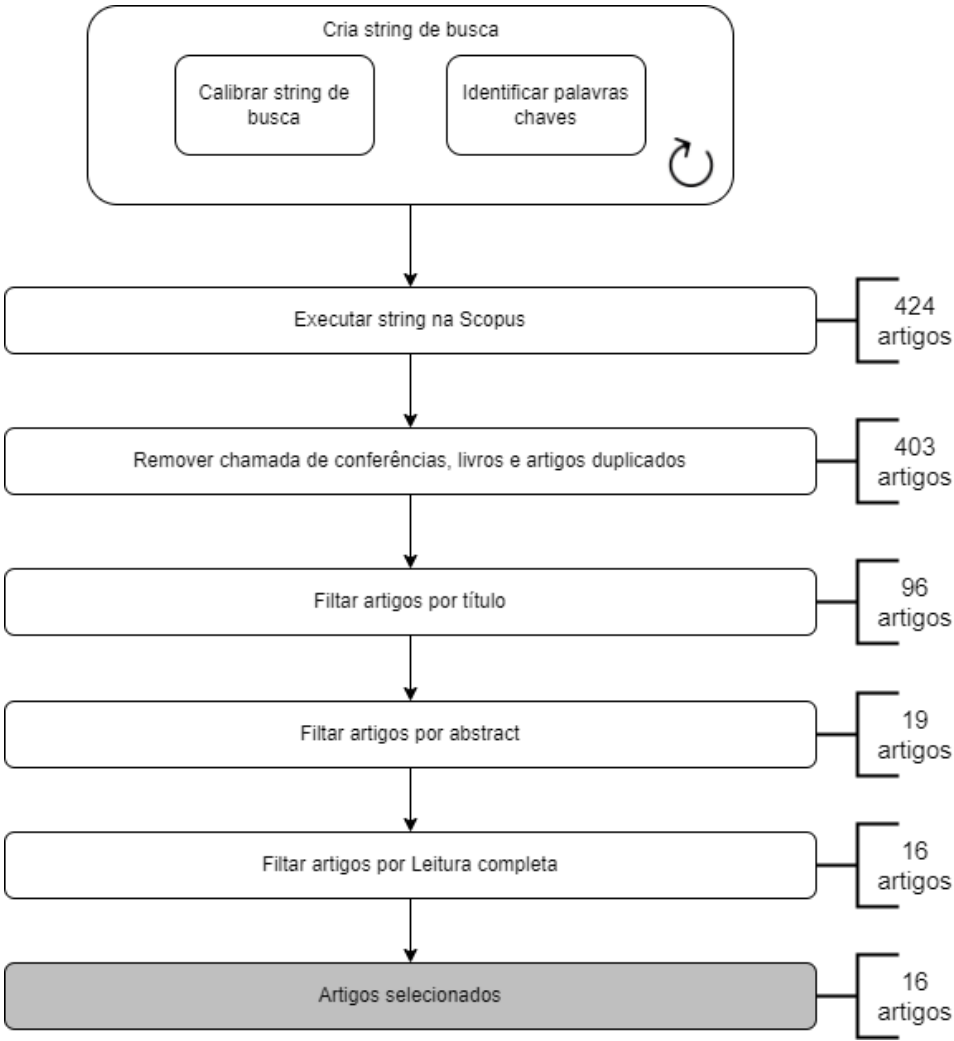


Figura 3: Protocolo de pesquisa aplicado. Elaborado pelo autor.

Atividade	Data	Número de artigos
Execução da string de busca	11/04/2024	424
Remover chamada de conferências	12/04/2024	403
Remover livros	12/04/2024	403
Remover duplicatas	12/04/2024	403
Filtro por título	12/04/2024	96
Filtro por resumo	22/06/2024	19
Filtro por leitura completa	19/07/2024	16
Total de artigos selecionados	19/07/2024	16

Tabela 4: Quantitativa de artigos em cada etapa da pesquisa.

Ao executar a string formulada, foram obtidos 424 artigos. Para poupar tempo e esforço desnecessário, esse número foi reduzido por meio da aplicação de filtros e remoção de artigos irrelevantes para a pesquisa. Removendo os livros, chamadas de conferências e duplicatas,

restaram 403 artigos. Livros são removidos devido aos seus tamanhos e o tempo de leitura que seria demandado, de acordo com [?]. Após isso, foram aplicados os filtros para reduzir ainda mais esse número, começando pelo filtro de títulos, onde após a execução, sobraram 96 artigos. O filtro por resumo diminuiu ainda mais esta quantia, trazendo esse número para 19. Por fim, foi feita a leitura completa dos 19 artigos remanescentes, que concluiu a sintetização dos artigos, excluindo mais 3, sobrando apenas 16 artigos que irão compor este trabalho.

3.1 Quais motivações são mencionadas para a criação de aplicativos para seleção de filmes em grupos?

A escolha de um filme para assistir em grupo pode ser um desafio significativo devido às diferenças de preferências individuais, influenciadas por fatores como personalidade, hábitos de consumo de mídia e experiências prévias com determinados gêneros cinematográficos. Esse problema se torna ainda mais evidente em grupos compostos por pessoas com gostos variados, como casais, amigos e familiares, onde a decisão precisa levar em conta múltiplas perspectivas. Aplicativos como o Filmy surgem para mitigar esse problema, utilizando métodos que facilitam a tomada de decisão e tornam a experiência mais interativa e envolvente. A motivação central para o desenvolvimento desses aplicativos está na indecisão causada pela diversidade de preferências, um fenômeno exemplificado no trecho:

"[...] indecisão também faz parte desse processo de escolha quando casais assistem filmes e séries juntos na plataforma, visto que 40% dos casais divergem no momento da decisão."[?]

Além da questão da indecisão, outro fator motivador para a criação desses aplicativos é a sobrecarga de opções nas plataformas de streaming. Com catálogos cada vez maiores, os usuários enfrentam dificuldades para encontrar conteúdos que atendam às suas expectativas, o que pode levar à chamada "paralisia da escolha"[?]. Esse fenômeno ocorre quando o excesso de alternativas leva à indecisão e, em alguns casos, ao abandono da busca. Aplicativos desse tipo podem explorar elementos de recomendação personalizados, ajudando a reduzir o tempo gasto na busca por conteúdos relevantes. O uso de inteligência artificial e aprendizado de máquina permite que esses aplicativos analisem preferências anteriores dos usuários e sugiram filmes compatíveis com os gostos do grupo. Dessa forma, a experiência de seleção se torna mais eficiente, interativa e social, reforçando o engajamento dos usuários com a plataforma.

Outra motivação relevante para a criação desses aplicativos está na valorização da experiência compartilhada. O ato de assistir a um filme não se resume apenas ao conteúdo consumido, mas também à experiência coletiva e à interação entre os espectadores. Aplicativos que ajudam na escolha do filme podem contribuir para tornar esse momento mais fluido e agradável, reduzindo frustrações e aumentando a satisfação do grupo com a decisão final. Portanto, a existência de aplicativos como Filmy e Queue responde a uma demanda crescente por soluções que auxiliem na escolha coletiva de filmes, tornando esse processo mais dinâmico, justo e menos desgastante.

3.2 Quais características os aplicativos encontrados utilizam para seleção de filmes em grupos?

Cada aplicativo voltado para a escolha de filmes em grupo adota estratégias específicas para facilitar a decisão e tornar a experiência do usuário mais fluida e interativa. No caso do Filmy, dois elementos principais são utilizados: a criação de personas e a gamificação. A criação de personas tem o objetivo de classificar o perfil do usuário com base em seu comportamento e preferências cinematográficas. Esse sistema permite que os usuários sejam identificados de acordo com categorias como explorador, para aqueles que gostam de descobrir novos gêneros e filmes menos conhecidos; e indeciso, que têm dificuldade em tomar decisões e precisam de recomendações externas. Com essa categorização, o aplicativo pode oferecer sugestões mais alinhadas ao perfil do grupo, reduzindo o tempo necessário para chegar a um consenso sobre qual filme assistir.

A gamificação é outro recurso essencial no Filmy, pois adiciona elementos interativos que incentivam os usuários a se engajarem com o aplicativo. Esse conceito é implementado por meio de três componentes principais: avatares, conquistas e recompensas. As conquistas podem ser obtidas ao atingir determinadas metas dentro do aplicativo, como assistir a um número específico de filmes dentro de um gênero ou finalizar uma maratona de filmes sugeridos. Como recompensa, os usuários podem desbloquear novos avatares baseados em personagens icônicos do cinema, tornando a experiência mais envolvente e personalizada [?].

Outro aplicativo relevante para a escolha de filmes em grupo é o Queue, que adota uma abordagem diferente ao priorizar um sistema de seleção democrático e colaborativo. No Queue, cada membro do grupo pode adicionar filmes à lista de sugestões, e o aplicativo usa um sistema

de votação para determinar qual será escolhido. Esse modelo resolve conflitos de interesse de forma transparente e justa, garantindo que todos os membros do grupo tenham voz na decisão. Além disso, o Queue permite a integração com serviços de streaming, facilitando o acesso imediato ao filme escolhido sem necessidade de alternar entre plataformas.

Além das estratégias específicas do Filmy e do Queue, aplicativos desse tipo podem incorporar outras funcionalidades, como recomendações baseadas em histórico de visualização, uso de filtros personalizados que permitem restringir sugestões com base em gênero, duração e classificação etária, bem como a sincronização entre dispositivos, possibilitando que membros do grupo participem da decisão remotamente. A combinação desses recursos torna a experiência de escolha mais intuitiva e divertida, evitando longas discussões e tornando a decisão coletiva mais eficiente. Na Tabela 5 podemos ver comparações feitas com as funcionalidades dos aplicativos Filmy e Queue, em comparação com o Match Movie.

	Gamificação	Votação	Criação de Personas	Filtros	Plataforma de Streaming Disponíveis	Redirecionamento para cinema	Calculo de localização
Filmy	✓	X	✓	✓	✓	X	X
Queue	X	✓	X	✓	✓	X	X
Match Movie	X	✓	X	✓	✓	✓	✓

Tabela 5: Matriz de rastreabilidade

Capítulo 4

Match Movie

4.1 Estratégia

Para o desenvolvimento do aplicativo, foi escolhido um desenvolvimento por etapas, que consiste em: identificação da ideia principal, elaboração do conceito visual, identificação da problemática a ser resolvida, aplicação do método de pesquisa *Rapid Review* e desenvolvimento do Match Movie.

4.2 Escopo

No escopo, serão definidas as funcionalidades, que estão divididas entre requisitos funcionais e não funcionais. Requisitos funcionais podem ser compreendidos como funcionalidades específicas que o sistema deve ser capaz de realizar, ou seja, descrevem as funcionalidades e comportamentos que o software deve apresentar para atender às necessidades dos usuários [?]. Já os requisitos não funcionais referem-se a características fora do olhar do usuário, como desempenho, segurança, acessibilidade, etc. [?]

4.2.1 Funcionalidades e requisitos

Requisitos funcionais

A coleta de requisitos é um processo fundamental no desenvolvimento de qualquer software, no entanto, o presente projeto não apresenta uma persona específica para delimitar as necessidades do projeto. Para resolver esse problema, todos os requisitos foram definidos com base nas necessidades encontradas por meio da revisão da literatura.

RF01 - Login. O aplicativo deve requisitar a todos os usuários login e senha antes de fornecer acesso às suas funcionalidades.

RF02 - Barra de navegação. O aplicativo deve conter uma barra de navegação na parte inferior de todas as telas.

RF03 - Perfil. O aplicativo deve permitir ao usuário clicar na aba de perfil. Nela, deverá ser permitido: editar nome, editar usuário, editar e-mail, editar número de celular, editar foto e desconectar da aplicação.

RF04 - Catálogo. O aplicativo deve permitir ao usuário: pesquisar filmes por nome, ler informações sobre qualquer filme e aplicar filtros de gênero, como: popular, em breve, em cartaz e mais votados.

RF05 - Match. O aplicativo deve permitir ao usuário entrar na aba do match. A aba do match deve conter as sessões já criadas, selecionar suas sessões e criar novas sessões.

RF07 - Sessões. O aplicativo deve permitir ao usuário clicar em suas sessões existentes ou criar uma nova sessão. O usuário pode: editar o nome, editar a foto, adicionar usuários, editar filtros da sessão, editar filme de referência, iniciar match e ver histórico.

RF06 - Votação de filmes. O aplicativo deve permitir aos usuários, em uma sessão, votar em filmes disponíveis no catálogo, seguindo as regras e filtros configurados na sessão.

RF06 - Informações de filmes. O aplicativo deve permitir aos usuários ver informações básicas sobre os filmes durante a votação.

RF08 - Histórico de match. O aplicativo deve manter um histórico referente a cada match feito em cada sessão. Assim como também deve permitir ao dono do grupo excluir matches do histórico.

RF09 - Cálculo de localização. Com a votação concluída, caso tenha sido feita com o filtro 'em cartaz', o aplicativo deve calcular o ponto médio em coordenadas de latitude e longitude, dos usuários, baseado no CEP de cada membro do grupo, que foi preenchido no formulário de cadastro.

RF10 - Redirecionamento para o Ingresso.com. Após a conclusão da votação, o aplicativo deve permitir ao usuário o redirecionamento para o cinema mais próximo do ingresso.com, baseando-se nas coordenadas fornecidas pela API do *OpenCage* e levando os usuários do grupo para a página do filme dentro do site ou aplicativo.

Requisitos não-funcionais

RNF01 - Verificação de login. O aplicativo deve verificar se o usuário já está logado ao iniciar a aplicação. Caso esteja, o aplicativo não exibirá a tela de login e irá automaticamente para o catálogo após a tela de apresentação. Caso o usuário não esteja logado, a aplicação irá para a tela de login após a tela de apresentação.

RNF02 - Versão do Android. O aplicativo deve funcionar corretamente a partir da versão 10 do Android.

RNF03 - Conformidade com a LGPD. A aplicação deve seguir as diretrizes estabelecidas pela Lei Geral de Proteção de Dados (LGPD), garantindo a privacidade e a segurança das informações pessoais dos usuários.

4.2.2 Diagramação

A seção de diagramação apresenta os principais diagramas utilizados no desenvolvimento do Match Movie, fornecendo uma visão estruturada sobre o funcionamento e a arquitetura do sistema. Esses diagramas ajudam a compreender como as partes do aplicativo interagem entre si e como os casos de uso foram projetados para atender aos requisitos levantados durante a análise inicial do projeto. Os diagramas apresentados incluem:

Diagrama de Caso de Uso: Detalha as principais interações entre os usuários e o sistema, destacando funcionalidades como a votação de filmes, o redirecionamento para o ingresso.com e o redirecionamento para serviços de *streaming*. Apêndice 20.

Diagrama de Sequência: Representa o fluxo de comunicação entre as diferentes camadas do sistema, ilustrando como as ações dos usuários percorrem o *front-end*, *back-end* e serviços externos, como a API do Ingresso.com e a TMDb. Foram elaborados 5 diagramas, abordando os principais pontos do sistema, que são: fluxo de login de usuário (Apêndice 21), registro de usuário (Apêndice 22), obtenção de filmes pelo TMDb (Apêndice 23), votação de filmes (Apêndice 25) e marcação de filmes como favorito e/ou assistido (Apêndice 24).

Diagrama de Classe: Descreve a estrutura do sistema em termos de classes, seus atributos e métodos, refletindo o modelo de domínio do aplicativo. Esse diagrama destaca elementos-chave, como usuários, filmes e sessões de votação, evidenciando as relações entre eles. Apêndice 19.

Esses diagramas desempenham um papel fundamental na documentação do projeto, garantindo que todos os membros da equipe compreendam a lógica de implementação e facilitando futuras manutenções ou expansões do sistema. Além disso, eles servem como uma base para validação das decisões arquitetônicas tomadas ao longo do desenvolvimento.

4.3 Telas

Nesta seção, serão apresentadas as telas do aplicativo, que foram desenvolvidas e construídas utilizando o programa Figma. A aplicação conta com uma barra de navegação na parte inferior da tela.

4.3.1 Tela de Apresentação

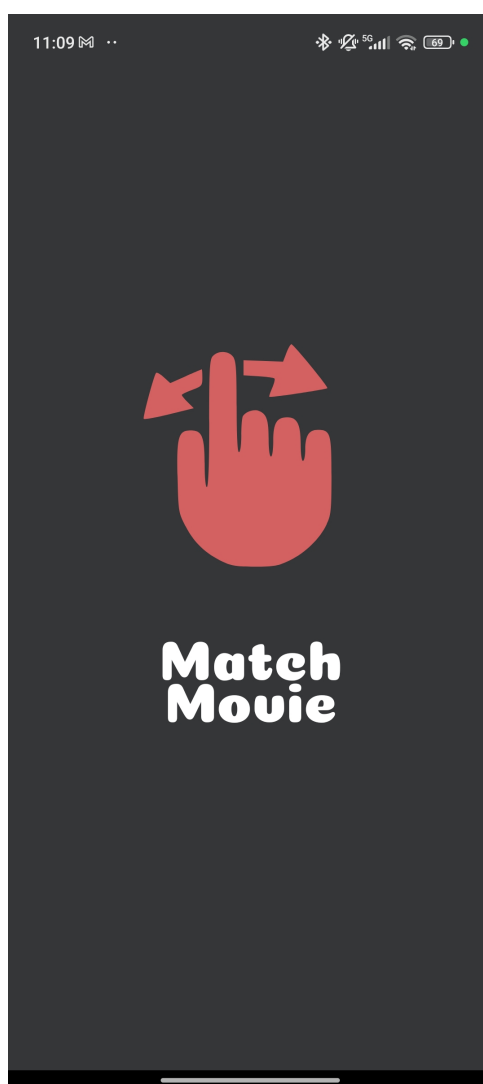


Figura 4: Primeira tela ao abrir o aplicativo. Elaborado pelo autor.

O primeiro contato com o aplicativo será a tela de apresentação, conhecida como *Splash Screen*. Ela não é uma tela interativa, nesta tela apenas aparecerá o logo da aplicação para o carregamento correto do aplicativo e verificação para caso o usuário já esteja logado. Caso

esteja, irá transitar para a tela de catálogo; caso não, a tela mudará de maneira fluida para a tela de login. Figura 4.

4.3.2 Tela de Login e Registro

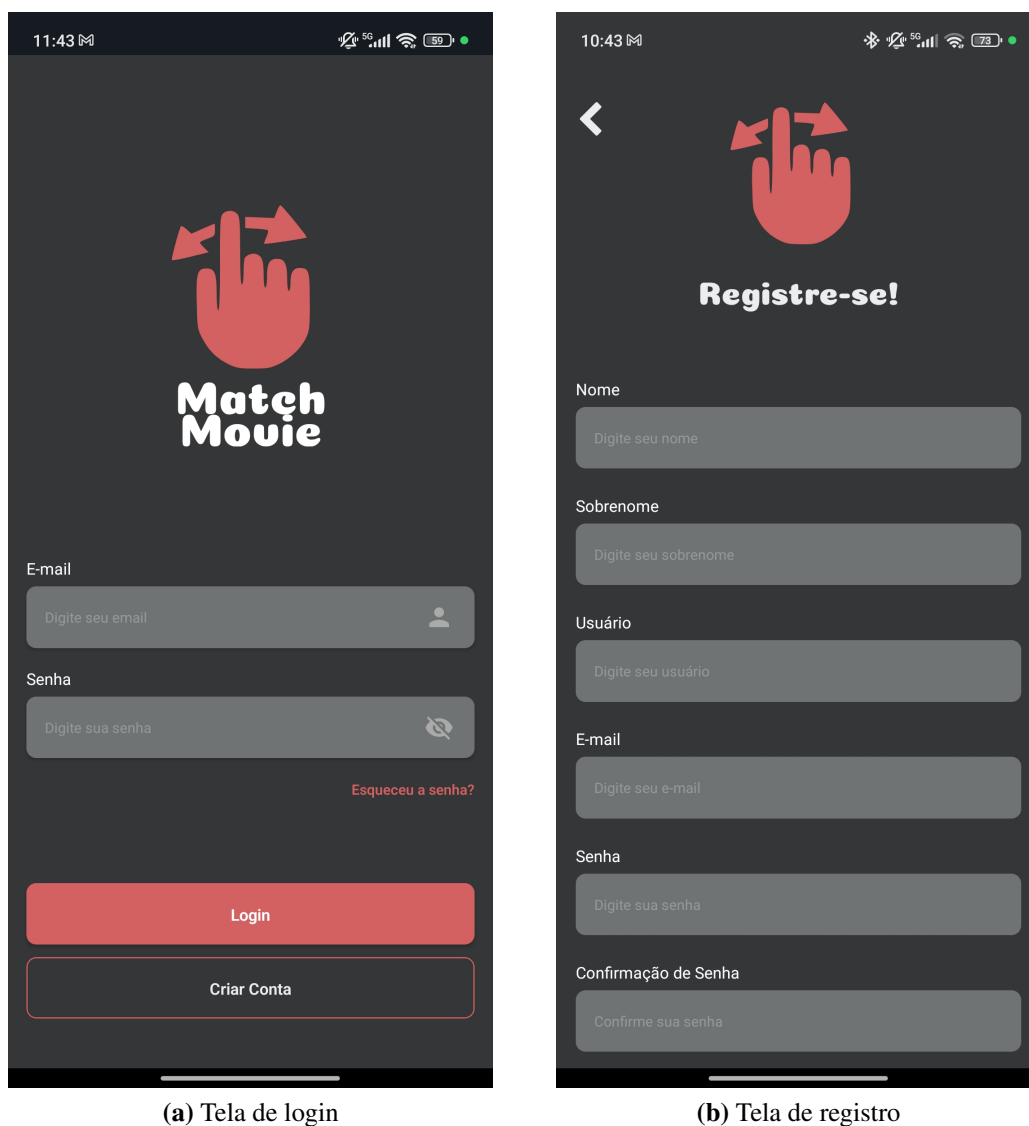


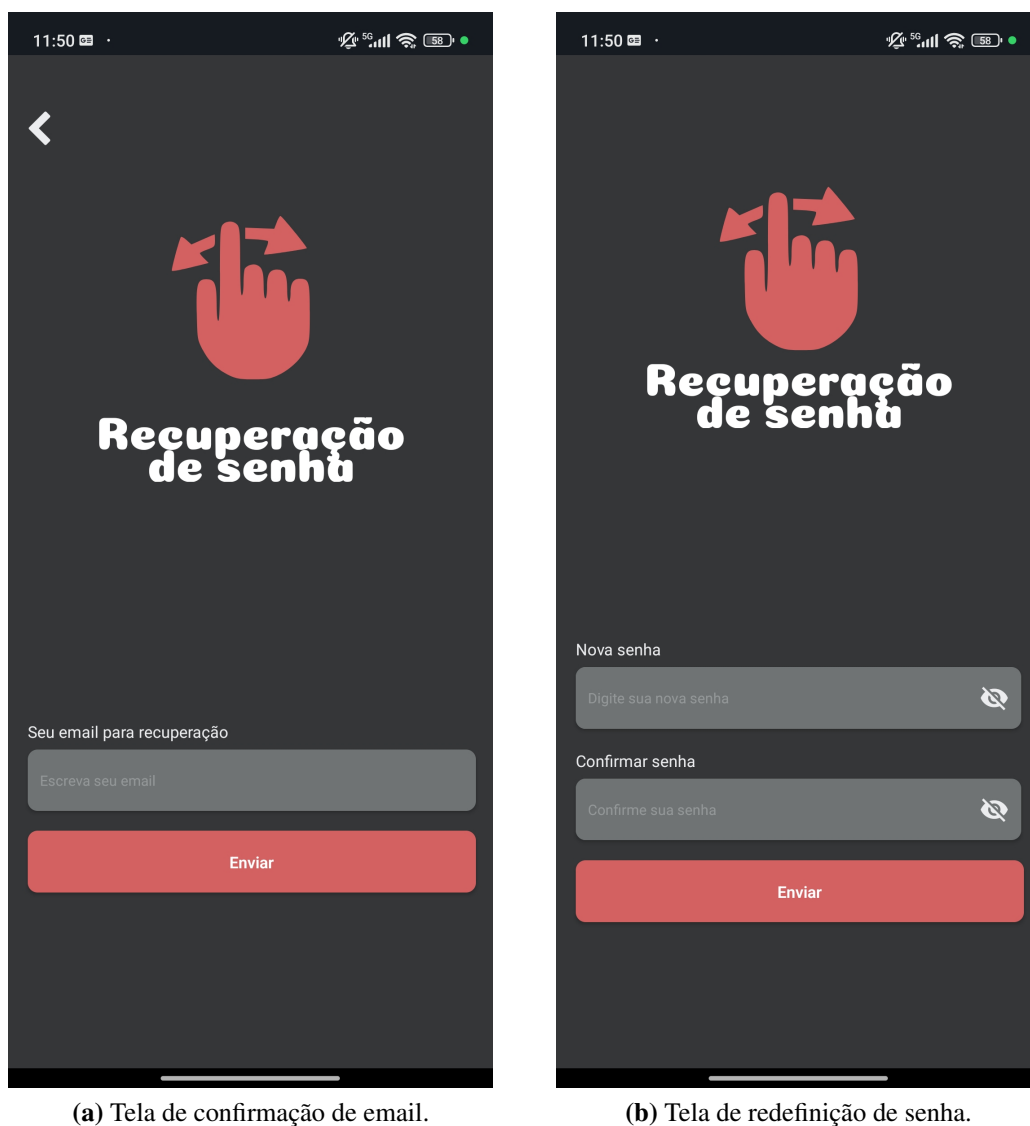
Figura 5: Telas de login e de registro. Elaboradas pelo autor.

Na tela de login, o usuário deve preencher com suas informações caso já tenha se registrado previamente. Caso não, o usuário deverá clicar em "Registre-se" para ser redirecionado para a tela de registro. Esta tela aparecerá somente caso o usuário não tenha feito login/se registrado, caso contrário, o login será feito automaticamente e seguirá para a próxima tela.

Caso o usuário tenha clicado em registrar-se, essa será a tela para a qual ele será redireci-

onado, precisando preencher os campos nela indicados para se registrar na plataforma corretamente. Figura 5.

4.3.3 Telas de recuperação de senha



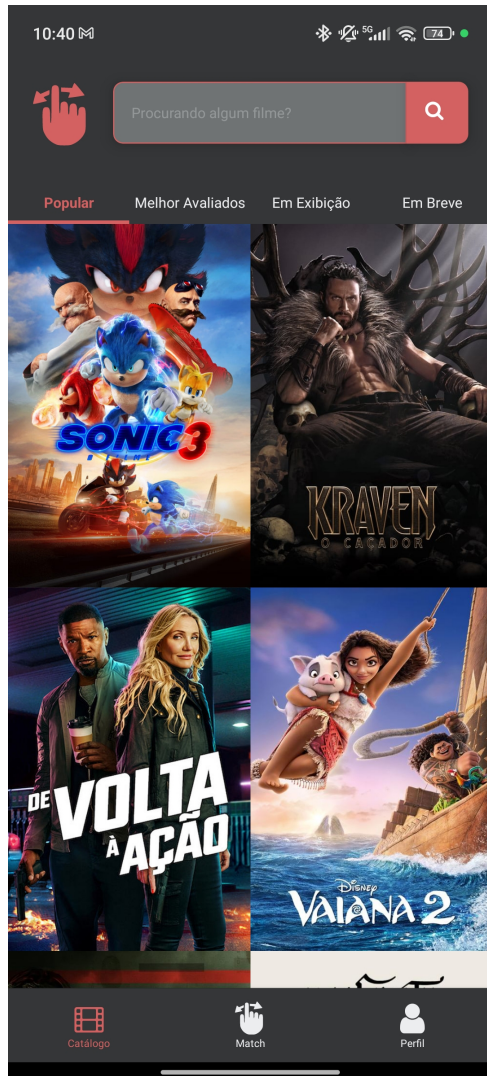
(a) Tela de confirmação de email.

(b) Tela de redefinição de senha.

Figura 6: Telas de redefinição de senha. Elaboradas pelo autor.

Caso o usuário, na tela de login, tenha clicado em “Esqueceu a senha?”, o aplicativo irá redirecioná-lo para a tela de confirmação de email. Assim que o email for confirmado e o usuário clicar no link que recebeu, será redirecionado para a segunda tela de recuperação, onde irá atualizar sua senha. 6

4.3.4 Tela de Catálogo



(a) Tela de catálogo dos filmes



(b) Tela de informações dos filmes do catálogo

Figura 7: Tela de catálogo dos filmes e tela de informações. Elaboradas pelo autor.

A tela de catálogo (primeira tela da barra de navegação) apenas mostrará todos os filmes dentro do filtro "Populares", que é pré-selecionado automaticamente. O usuário poderá aplicar outros filtros, buscar por nome e clicar em algum filme para ler sobre suas informações.

Caso o usuário tenha clicado em algum filme na tela de catálogo, ele será movido para esta tela, onde poderá ler sobre algumas informações sobre o filme, como: título, nota, plataformas onde ele está disponível, elenco, etc. Aqui também será possível marcar o filme como visto ou como favorito. Figura 5.

4.3.5 Tela de Perfil

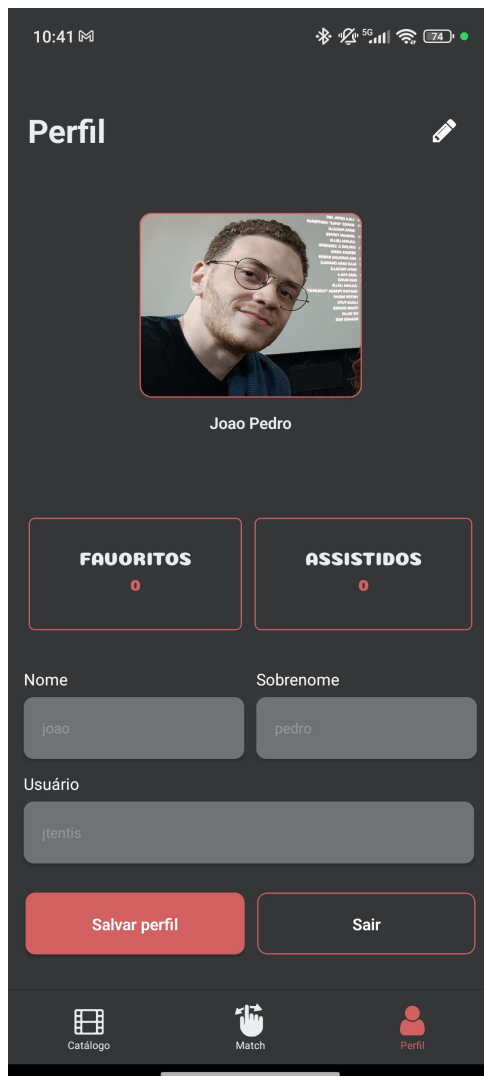


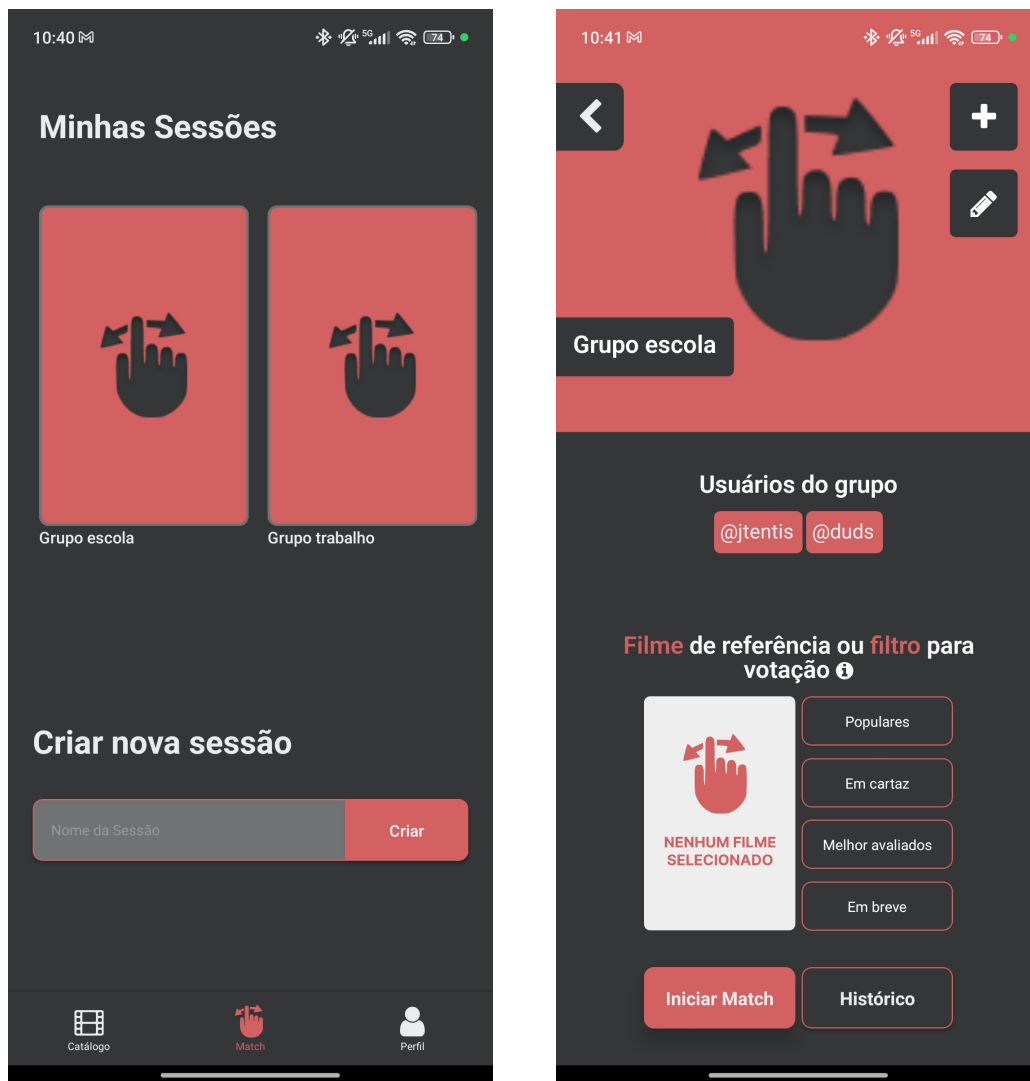
Figura 8: Tela de perfil do usuário. Elaborada pelo autor.

Nesta tela (segunda tela da barra de navegação), o usuário poderá ver/alterar algumas de suas informações, alterar a imagem de perfil e desconectar da aplicação. Caso o usuário desconecte, ele será redirecionado para a tela de login. Figura 8.

4.3.6 Tela de Match - Sessões

A tela de match (segunda tela na barra de navegação) é onde o usuário verá suas sessões/grupos, podendo clicar nas suas sessões já criadas, entrar em uma sessão já existente por meio do link ou criar uma nova sessão.

Caso o usuário tenha clicado em alguma de suas sessões, ele será redirecionado para esta tela. Caso ele seja o dono da sessão, ele poderá: editar o nome, escolher filmes de referência (máximo de 3 filmes), ativar e desativar filtros, ver o histórico da sessão e iniciar a votação. Figura 9.



(a) Primeira tela de match, onde o usuário vê suas sessões

(b) Tela quando o usuário selecionar alguma de suas sessões/grupos

Figura 9: Tela de match e tela de sessão. Elaboradas pelo autor.

4.3.7 Tela de Match - Histórico

Caso o usuário tenha clicado em "histórico" na tela anterior, ele será movido para esta tela, onde poderá ver os Matches que sua sessão já fez, podendo excluir Matches anteriores. Figura 10.

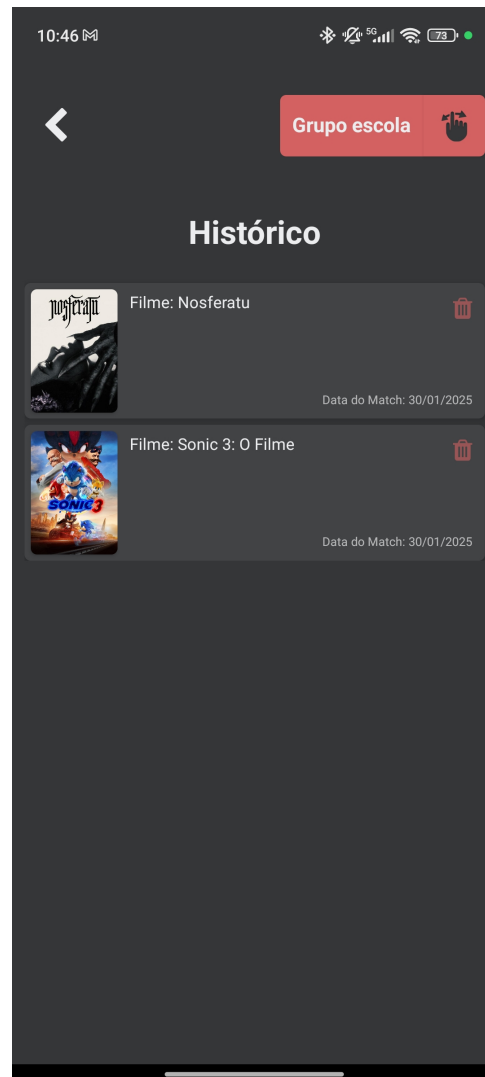
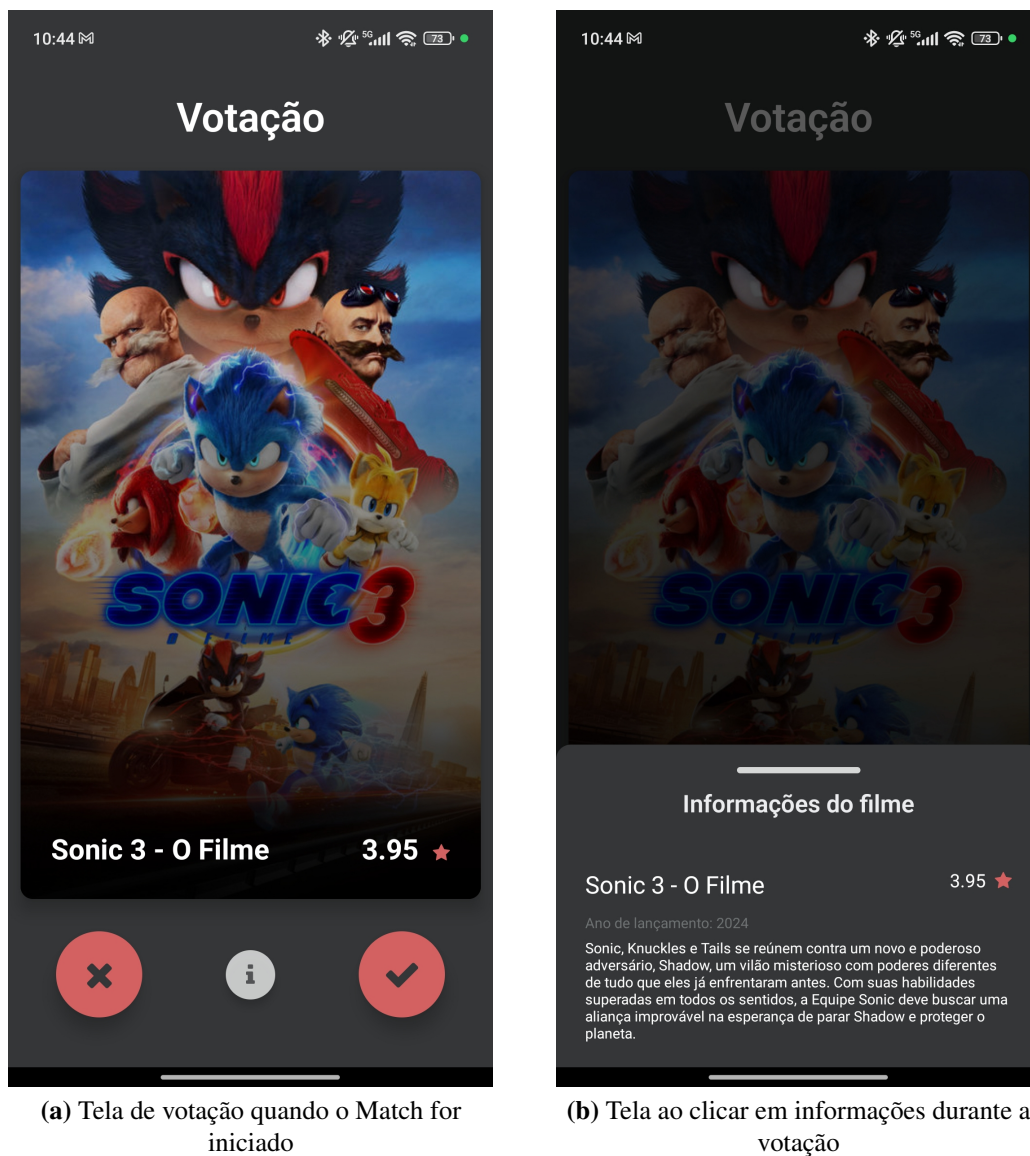


Figura 10: Tela para o usuário ver os históricos de suas sessões. Elaborada pelo autor.

4.3.8 Tela de Match - Votação



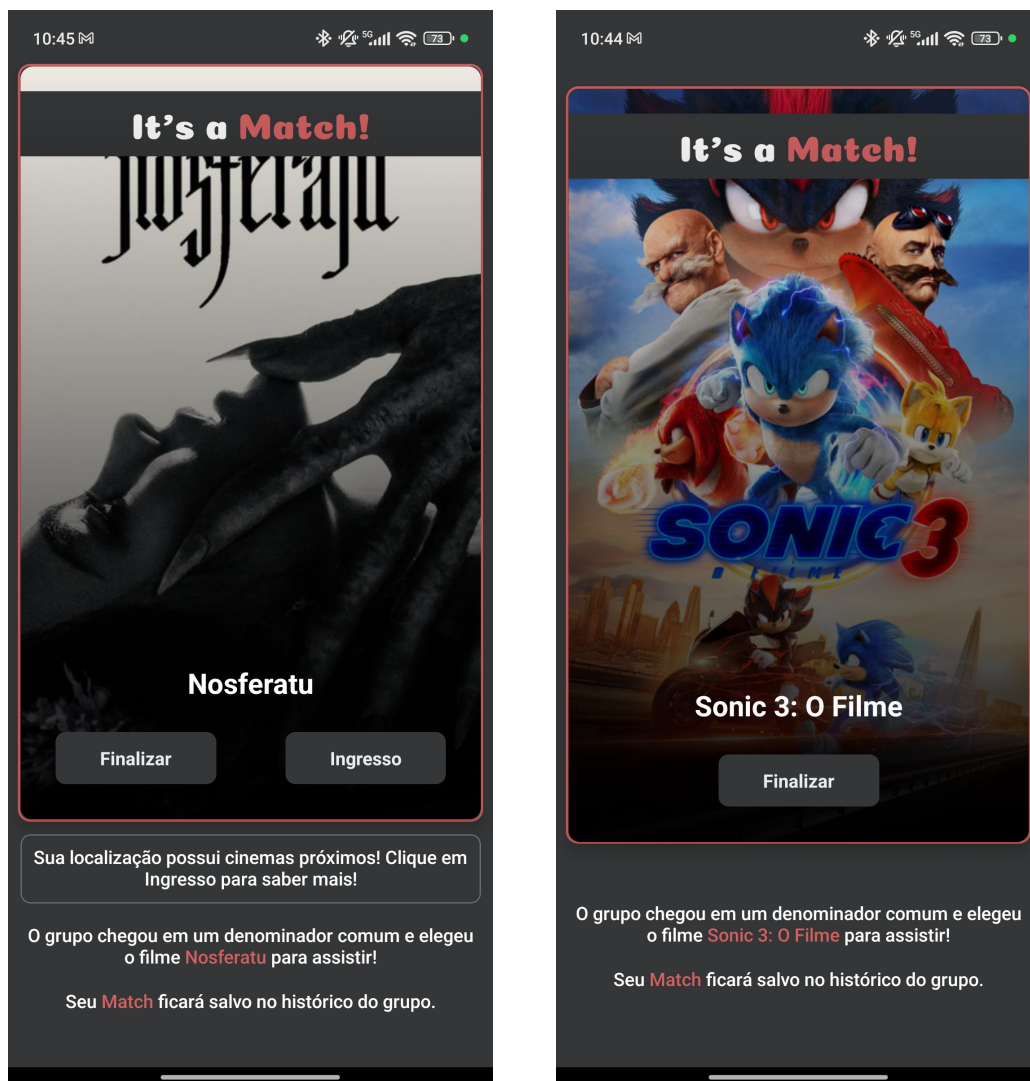
(a) Tela de votação quando o Match for iniciado

(b) Tela ao clicar em informações durante a votação

Figura 11: Tela de votação e pop-up de informação. Elaboradas pelo autor.

Nesta tela, que aparecerá quando a votação for iniciada, os usuários do grupo deverão votar nos filmes, podendo arrastar para os lados ou clicar nos símbolos. Caso o usuário clique no símbolo do meio de “informação”, um *modal* irá abrir com as informações pertinentes àquele filme. Figura 11.

4.3.9 Tela de Match - Conclusão de votação



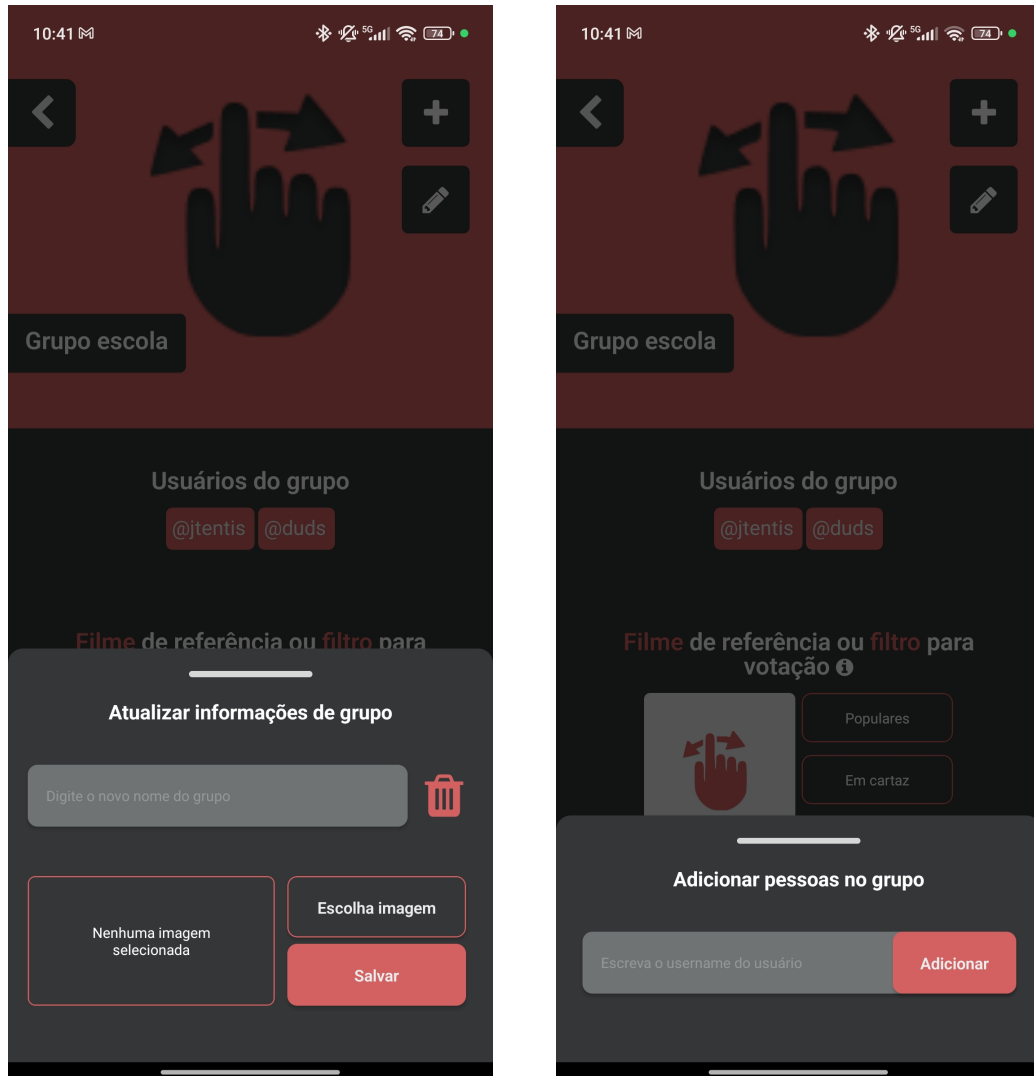
(a) Tela de conclusão com redirecionamento.

(b) Tela de conclusão sem redirecionamento.

Figura 12: Telas de conclusão de votação. Elaboradas pelo autor.

A tela de conclusão de votação tem 2 tipos: votação concluída utilizando o filtro dos filmes “Em cartaz” (Figura (a)) e votação concluída utilizando os demais filtros ou filme de referência (Figura (b)). Figura 12.

4.3.10 *Modals*



(a) *Modal* para edições em grupo. Presente na tela da Figura 9.

(b) *Modal* para adições de membros em grupo. Presente na tela da Figura 9.

Figura 13: *Modals* presentes nas telas de grupo. Elaboradas pelo autor.

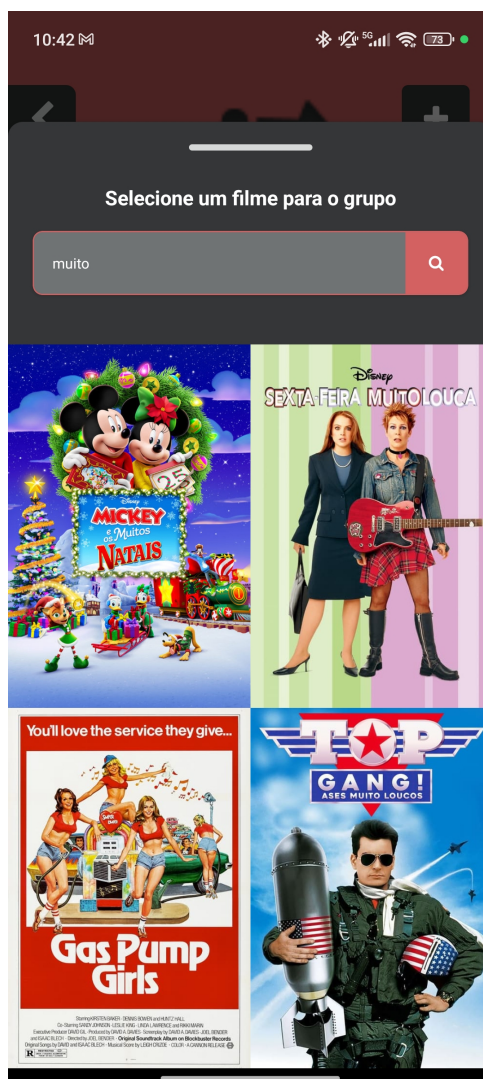


Figura 14: *Modal* para escolher filme de referência para grupo. Presente na tela da Figura 9.

Por fim, o aplicativo conta com alguns *modals*, que são pequenas telas que aparecem quando o usuário completa alguma ação ou clica em algum botão. Os principais são: *modal* para quando um usuário quiser adicionar membros em um grupo, Figura (a) referenciada na Figura 13; *modal* para quando um usuário quiser editar nome, editar imagem ou excluir um grupo, Figura (b) referenciada na Figura 13 e *modal* para quando um usuário for escolher um filme de referência para o grupo, Figura 14.

4.4 Marca

A construção da marca do Match Movie foi pensada para transmitir a essência do aplicativo de maneira clara, envolvente e funcional. Cada elemento que compõe a marca reflete seu propósito principal: oferecer uma experiência simples e intuitiva que conecta grupos na escolha do

filme ideal.

A identidade da marca combina elementos visuais e conceituais cuidadosamente projetados, garantindo coerência e harmonia em todos os aspectos do aplicativo. O logotipo, o *naming*, a tipografia e as escolhas de cores foram integrados para criar uma identidade visual que se destaque, seja memorável e, ao mesmo tempo, funcional para o público-alvo.

Nos subcapítulos a seguir, serão explorados os detalhes que compõem a marca, desde o conceito por trás do logotipo até as decisões sobre tipografia e *naming*. Essas escolhas não apenas fortalecem a identidade visual do aplicativo, mas também garantem uma experiência alinhada aos valores e objetivos do Match Movie.

4.4.1 Paleta de cores

O design do Match Movie utiliza duas cores principais: #454542 (um tom de cinza-escuro) e #C3635B (um tom de vermelho terroso). A escolha dessas cores foi feita utilizando o website *Pigment Shapefactory*, como visto nas figuras 15a e 15b e com base nos seguintes critérios:

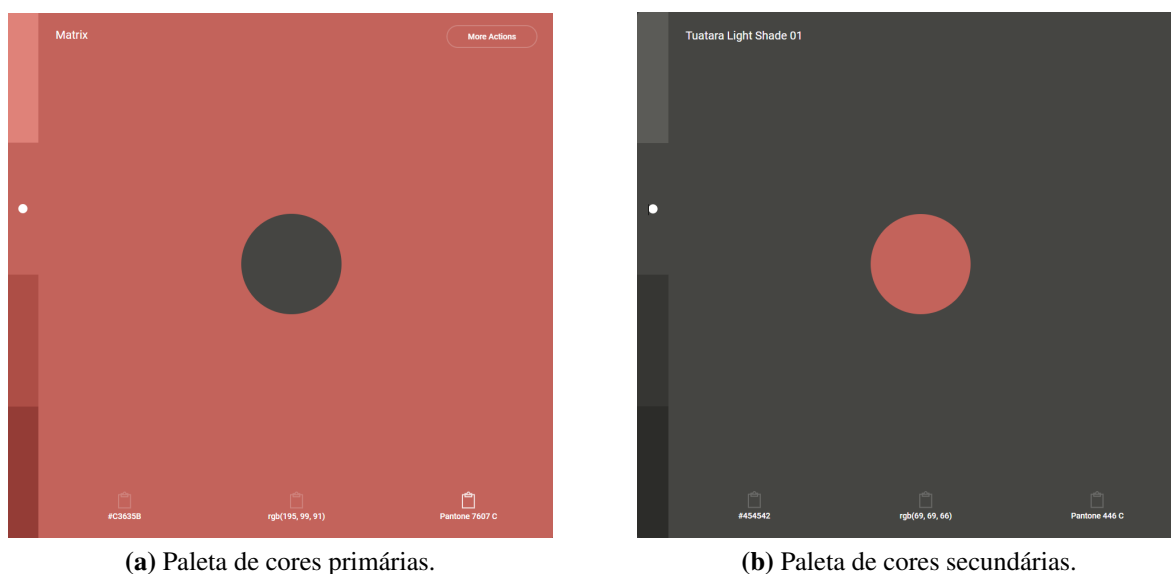


Figura 15: Paletas de cores primárias e secundárias. Fonte: *Pigment Shapefactory*.

4.4.2 Logotipo

O logotipo do aplicativo foi projetado para refletir a essência do processo de interação que ele proporciona aos usuários. Inspirado na funcionalidade principal do aplicativo, o desenho apresenta um dedo simulando o movimento de "arrastar" para a esquerda e para a direita, re-

metendo de maneira intuitiva à mecânica de escolha utilizada pelos usuários, semelhante ao modelo de aplicativos como o Tinder. Esse movimento simboliza a simplicidade e praticidade da votação de filmes em grupo, elemento central da proposta do aplicativo.

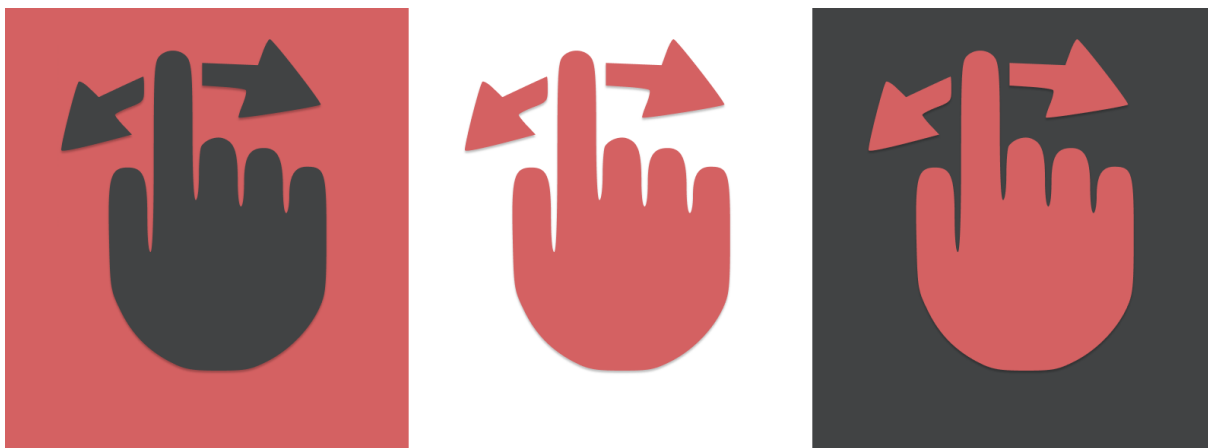


Figura 16: Logotipos com as cores usadas no aplicativo. Elaborado pelo autor.

A paleta de cores do logotipo reforça a identidade visual do aplicativo. Como discutido no tópico sobre paleta de cores, o aplicativo utiliza como cor primária o tom vibrante de (#C3635B), que transmite energia e dinamismo. A cor secundária, (#454542), adiciona equilíbrio e sofisticação, criando um contraste harmonioso. Além disso, o logotipo também está disponível em versão branca, garantindo versatilidade para diversas aplicações e contextos. Figura 16.

No design, a cor primária é utilizada predominantemente quando o fundo do aplicativo assume a cor secundária, e vice-versa, criando uma experiência visual consistente e agradável. Essa alternância de cores reflete a flexibilidade e adaptabilidade do logotipo, destacando-o de maneira elegante em diferentes cenários, sejam digitais ou impressos.

Ao unir uma estética moderna com uma funcionalidade clara e objetiva, o logotipo reforça o propósito do aplicativo: simplificar escolhas em grupo de forma interativa e eficiente. Ele não apenas simboliza a funcionalidade do aplicativo, mas também cria uma conexão visual imediata com os usuários, tornando-se um elemento memorável e representativo da marca.

4.4.3 Escolha do nome

O nome do aplicativo, Match Movie, foi cuidadosamente escolhido para sintetizar as duas ideias principais que guiam sua funcionalidade e propósito. A combinação das palavras “Match” e “Movie” representa a essência do aplicativo: unir pessoas em torno da escolha de um filme,

promovendo uma experiência colaborativa e divertida.

O termo "Movie"reflete o vasto catálogo de filmes disponíveis na plataforma, que serve como base para as votações. Já a palavra "Match"simboliza o objetivo final do aplicativo: encontrar a melhor combinação de preferências entre os usuários, promovendo um consenso na escolha do filme ideal. Assim, o nome não apenas comunica a funcionalidade principal, mas também transmite de forma clara e direta a proposta de valor do aplicativo.

Ao unir conceitos simples e universais, o nome Match Movie cria uma identidade forte e facilmente reconhecível, ressoando com os usuários que buscam praticidade e interatividade ao decidir o que assistir. O nome também reflete a abordagem inclusiva e dinâmica do aplicativo, ao transformar um processo que muitas vezes é marcado por indecisões em uma atividade leve, eficiente e social.

Essa escolha de *namíng* reforça o posicionamento do aplicativo como uma solução inteligente para grupos que desejam aproveitar momentos de entretenimento juntos, minimizando conflitos e maximizando a experiência compartilhada. Com um nome fácil de lembrar e que encapsula sua proposta, Match Movie se apresenta como uma marca coesa, acessível e alinhada às expectativas de seu público-alvo.

4.4.4 Tipografia

A tipografia desempenha um papel crucial na construção da identidade visual e na usabilidade de um aplicativo. No desenvolvimento do Match Movie, foram selecionadas duas fontes que, em conjunto, equilibram personalidade e legibilidade: Coiny para o logotipo e Inter para os textos internos do aplicativo.

Coiny (Figura 17a e Figura 18), utilizada exclusivamente no logotipo, é uma fonte display com cantos arredondados que transmite uma sensação amigável e acessível. Desenvolvida originalmente por Manushi Sarin, Coiny apresenta formas baseadas em geometria simples e é inspirada em designs vernaculares urbanos [?]. Sua aparência remete à escrita com ponta de pincel arredondada, tornando-a especialmente eficaz para títulos e cabeçalhos que buscam chamar a atenção de maneira lúdica e envolvente.

Coiny

The quick brown fox jumps over the lazy dog

Aa Bb Cc Dd Ee Ff Gg Hh Ii Jj Kk Ll Mm

Nn Oo Pp Qq Rr Ss Tt Uu Vv Ww Xx Yy Zz

1234567890 (.,!/?#\$%&*'\@:;)

Penultimate

The spirit is willing but the flesh is weak

SCHADENFREUDE

3964 Elm Street and 1370 Rt. 21

(a) Figura de exemplo da fonte Coiny.

Inter

The quick brown fox jumps over the lazy dog

Aa Bb Cc Dd Ee Ff Gg Hh Ii Jj Kk Ll Mm

Nn Oo Pp Qq Rr Ss Tt Uu Vv Ww Xx Yy Zz

1234567890 (.,!/?#\$%&*'\@:;)

Penultimate

The spirit is willing but the flesh is weak

SCHADENFREUDE

3964 Elm Street and 1370 Rt. 21

(b) Figura de exemplo da fonte Inter.

Figura 17: Exemplos das fontes Coiny e Inter. Fonte: *Google Fonts*

Para os textos internos do aplicativo, a escolha recaiu sobre a fonte Inter Regular 17b, projetada por Rasmus Andersson. Inter é uma família tipográfica sem serifa que foi criada especificamente para telas digitais. Destaca-se pelo alto x-height, que melhora a legibilidade de textos em caixa baixa e mista, além de possuir aberturas amplas que aumentam a clareza dos caracteres em textos densos [?]. Estudos mostram que fontes sem serifa, como Inter, podem melhorar significativamente o desempenho de leitura em dispositivos digitais, especialmente em contextos onde a legibilidade é crítica [?].

A combinação de Coiny e Inter no Match Movie não apenas reforça a identidade visual do aplicativo, mas também assegura uma experiência de leitura agradável e eficiente para os usuários. Enquanto Coiny adiciona um toque de personalidade e distinção ao logotipo, Inter garante que os textos sejam facilmente legíveis, contribuindo para uma interface de usuário intuitiva e acessível.

4.5 Desenvolvimento

O desenvolvimento do Match Movie foi pautado em decisões estratégicas e tecnológicas que visaram criar uma solução eficiente, escalável e com uma experiência de usuário fluida. Desde a escolha das ferramentas até a implementação das funcionalidades, cada etapa foi pensada para

unir desempenho, praticidade e um ecossistema coeso.



Figura 18: Exemplo da fonte Coiny nas paletas de cores do aplicativo. Elaborado pelo autor.

Inicialmente, a escolha da linguagem e das tecnologias envolvidas levou em conta a curva de aprendizado e a compatibilidade entre as partes do sistema. Java foi a primeira opção para o desenvolvimento do aplicativo devido ao seu desempenho nativo em dispositivos Android. Contudo, após a primeira versão do aplicativo, percebeu-se a necessidade de unificar o ecossistema do projeto, já que a API já havia sido desenvolvida em Node.js, um framework robusto e eficiente para o **back-end**, que utiliza JavaScript como base. A transição para React Native no **front-end** permitiu que o aplicativo utilizasse a mesma linguagem no **back-end** e no **front-end**, facilitando o desenvolvimento e a manutenção do código. O React Native, além de possibilitar o desenvolvimento multiplataforma, entregou uma experiência visual responsiva e amigável aos usuários.

No **back-end**, o uso do Node.js foi essencial para suportar múltiplas requisições simultâneas, característica indispensável para o sistema de votação em tempo real. O modelo de event loop do Node.js garantiu que o servidor pudesse lidar com diversas interações ao mesmo tempo, de forma eficiente, oferecendo escalabilidade e baixo consumo de recursos.

Para a camada de dados, o banco de dados PostgreSQL foi escolhido devido à sua confiabilidade, suporte a transações ACID e capacidade de lidar com estruturas complexas, como o armazenamento das preferências e histórico de votos dos usuários. O Prisma ORM foi utilizado como intermediário entre o Node.js e o PostgreSQL, simplificando a manipulação de dados e oferecendo uma abstração clara e segura do banco. Sua tipagem estática e suporte a migrações automatizadas facilitaram a manutenção do esquema de dados, garantindo maior agilidade no desenvolvimento.

A combinação dessas tecnologias resultou em um ecossistema coeso, no qual React Native, Node.js, PostgreSQL e Prisma ORM trabalharam juntos para oferecer uma aplicação escalável, eficiente e de fácil manutenção. O Match Movie foi concebido para atender a um público que busca praticidade e interação no processo de escolha de filmes em grupo, oferecendo uma

experiência intuitiva e consistente, graças às escolhas tecnológicas alinhadas aos objetivos do projeto.

4.5.1 Arquitetura da Aplicação

A arquitetura utilizada no desenvolvimento do **Match Movie** (Apêndice 26) segue o padrão do **Nest.js**, que organiza o código de maneira modular e escalável em quatro camadas principais: *Controllers*, *Services*, *Entities* e *DTOs*. Essa estrutura é amplamente adotada por suas vantagens em relação à manutenção, testabilidade e organização do projeto [?].

Controllers são responsáveis por gerenciar as requisições HTTP recebidas e direcioná-las para a camada de serviços apropriada. Eles definem as rotas e atuam como intermediários entre o cliente e a lógica de negócio da aplicação.

Services contêm a lógica de negócio da aplicação. Eles processam os dados recebidos dos *controllers*, interagem com a camada de dados por meio das entidades e retornam as respostas necessárias. A separação da lógica em serviços promove a reutilização do código e facilita os testes unitários.

Data Transfer Objects: Objetos de Transferência de Dados (DTO)s são utilizados para garantir que apenas os dados necessários sejam transferidos entre as camadas da aplicação. Além disso, no **Nest.js**, os DTOs desempenham um papel importante na validação dos dados de entrada, assegurando que as requisições estejam no formato esperado e evitando inconsistências.

A organização modular da arquitetura do **Nest.js** facilitou o desenvolvimento do **Match Movie**, permitindo uma separação clara das responsabilidades, escalabilidade da aplicação e uma maior eficiência no gerenciamento do código. Essa abordagem também tornou a aplicação mais robusta e alinhada com as melhores práticas de desenvolvimento modernas [?].

4.5.2 API

A API desempenha um papel fundamental na arquitetura do **Match Movie**, servindo como um intermediário robusto entre o *front-end*, desenvolvido em React Native, e os serviços externos consumidos. A decisão de construir uma API utilizando Node.js, separada do aplicativo principal, foi estratégica e alinhada às boas práticas modernas de desenvolvimento. Para garantir a segurança e a organização, todos os endpoints são protegidos por autenticação *Json Web Token (JWT)*, assegurando que apenas usuários autenticados possam acessá-los. Além disso,

foi implementado o **Swagger** para documentação da API, facilitando a visualização e o teste dos endpoints, com uma camada adicional de proteção utilizando **BasicAuth** para restringir o acesso ao Swagger apenas a usuários autorizados.

Para proporcionar uma visão mais detalhada dos endpoints disponíveis e suas respectivas funcionalidades, imagens ilustrativas do Swagger foram incluídas nos Apêndices ??, ?? e ??. Essas figuras permitem compreender de forma clara as operações disponíveis na API e a estrutura das requisições e respostas, auxiliando no entendimento e na manutenção do sistema.

Motivo da Construção da API em Node.js: A escolha do Node.js como tecnologia para a API foi motivada por sua eficiência em lidar com múltiplas requisições simultâneas, graças ao seu modelo de *event loop* assíncrono. Diferente de soluções tradicionais que utilizam múltiplas threads, o Node.js é capaz de gerenciar um grande volume de conexões de maneira não bloqueante, o que o torna ideal para aplicações com alta demanda de interatividade [?]. Além disso, o uso do Node.js permitiu unificar o ecossistema do projeto, com JavaScript no *back-end* e no *front-end*, facilitando a integração e manutenção do código.

Construção de uma API Intermediária: Em vez de consumir diretamente as APIs externas no código do React Native, optou-se por desenvolver uma **API intermediária**. Essa abordagem apresenta diversas vantagens, tanto técnicas quanto funcionais:

A API centraliza toda a lógica de requisições e manipulação de dados, filtrando e organizando as respostas provenientes das APIs externas, resultando em um código mais limpo e de fácil manutenção no *front-end*. Além disso, ela reduz o tráfego de dados ao filtrar informações irrelevantes das APIs externas, entregando apenas os dados essenciais e otimizando o desempenho da aplicação. Como uma camada intermediária, a API também atua na segurança, protegendo o *front-end* de informações sensíveis, como chaves de API e *endpoints*. Por fim, proporciona flexibilidade e facilidade de manutenção, permitindo a adaptação ou substituição de provedores externos sem impactar diretamente o *front-end*, com mudanças limitadas à camada intermediária.

No **Match Movie**, a API desenvolvida em Node.js organiza e filtra as informações recebidas das APIs externas, entregando ao aplicativo apenas os dados relevantes e no formato adequado. Essa abordagem proporciona um fluxo de dados mais limpo e eficiente, facilitando a renderização das interfaces no React Native e garantindo uma experiência de usuário rápida e responsiva. Além disso, a separação entre a API e o *front-end* contribui para uma arquitetura modular e escalável, simplificando a manutenção e evolução da aplicação.

Themoviedb.org

Para a extração de informações sobre os filmes, suas descrições e recomendações, foi utilizada a *The MovieDB (TMDB)*. A TMDB é uma plataforma popular e amplamente reconhecida por seu extenso banco de dados de filmes, séries de TV e informações relacionadas. Sua API fornece acesso gratuito a uma ampla gama de endpoints que permitem a busca, listagem e recomendação de filmes de forma dinâmica e eficiente [?].

No contexto do **Match Movie**, a API da TMDB desempenha um papel crucial no fornecimento dos dados necessários para as funcionalidades do aplicativo. Os principais recursos utilizados incluem:

A API é utilizada para buscar detalhes dos filmes, como título, descrição, ano de lançamento, gênero e avaliações, exibindo essas informações no aplicativo para auxiliar os usuários na votação. O filme selecionado pelo usuário como base é enviado para o endpoint de `/recommendations` da API da TMDB, que retorna uma lista de filmes recomendados com base no filme escolhido, fornecendo uma seleção filtrada e relevante para ser utilizada nas votações. Após a obtenção das recomendações, os filmes são sorteados de maneira dinâmica dentro da API intermediária construída em Node.js, garantindo uma seleção variada e imparcial e oferecendo uma experiência mais interessante e colaborativa para os usuários durante o processo de votação. Todos os endpoints podem ser vistos na Tabela 6.

Endpoint	Função
/movie/popular	Retorna uma lista dos filmes mais populares no momento.
/movie/top_rated	Retorna uma lista dos filmes mais bem avaliados.
/movie/now_playing	Retorna uma lista dos filmes que estão atualmente em cartaz nos cinemas.
/movie/upcoming	Retorna uma lista dos filmes que serão lançados em breve.
/search/movie?query={query}	Busca filmes com base em uma <i>string</i> informada pelo usuário.
/movie/{movieId}	Retorna detalhes de um filme específico, incluindo informações sobre elenco e equipe.
/movie/{movieId}/watch/providers	Retorna informações sobre onde o filme pode ser assistido.
/movie/{movieId}/recommendations	Retorna uma lista de filmes com base no filme do parâmetro.

Tabela 6: Endpoints utilizados para obtenção de informações de filmes. Fonte: *TheMovieDB.org*

Essa abordagem possibilita ao **Match Movie** oferecer uma experiência personalizada e eficiente ao usuário. A integração com a API da TMDb permitiu aproveitar uma base de dados robusta e atualizada, sem a necessidade de manutenção interna desse conteúdo, otimizando o desenvolvimento e garantindo a qualidade das informações apresentadas no aplicativo.

OpenCage Geolocation

Para calcular os cinemas mais próximos com base na localização dos usuários, o aplicativo utiliza a API da **OpenCage** para converter o CEP fornecido durante o cadastro em coordenadas geográficas. Esse processo é realizado para todos os membros do grupo, garantindo a obtenção precisa de suas respectivas localizações. A conversão é feita por meio do seguinte endpoint:

`https://api.opencagedata.com/geocode/v1/json?q={cep}`

A partir das coordenadas obtidas, o sistema calcula o ponto médio entre todos os membros do grupo, representando uma localização central que servirá como referência para a busca de cinemas. Esse ponto médio é então utilizado para consultar a API da **Ingresso.com**, identificando os cinemas mais próximos desse local.

Com essa abordagem, o aplicativo sugere locais de exibição que sejam convenientes para todos os participantes do grupo, otimizando a experiência do usuário ao facilitar a escolha de um cinema acessível para todos [?].

Ingresso.com

A integração do **Match Movie** com a API da **Ingresso.com** melhora a experiência do usuário ao conectar a votação de filmes com informações atualizadas sobre exibições nos cinemas. Quando um grupo finaliza a votação e chega a um consenso, ou seja, todos os participantes deram *"like"* no mesmo filme, o aplicativo verifica, por meio da API, se o filme está em cartaz.

Para isso, o sistema utiliza a geolocalização dos usuários e consulta a API da **Ingresso.com**. Primeiro, é feito um request para o endpoint:

```
https://api-content.ingresso.com/v0/states/city/latlong?lat={lat}&lng={lng}
```

Esse endpoint retorna o *cityId*, um identificador da cidade onde o usuário está localizado. Os valores de latitude (*lat*) e longitude (*lng*) são obtidos a partir da API da **OpenCage**, garantindo maior precisão na identificação da localização.

Com o *cityId* em mãos, a aplicação faz uma nova requisição ao seguinte endpoint:

```
https://api-content.ingresso.com/v0/templates/nowplaying/{cityId}
```

Esse endpoint retorna informações sobre os filmes atualmente em exibição nos cinemas mais próximos da cidade identificada. Caso o filme esteja em cartaz, o aplicativo exibe um redirecionamento para a página correspondente no site ou no aplicativo da **Ingresso.com**, permitindo que os usuários visualizem horários de exibição e realizem a compra dos ingressos de forma prática [?].

Essa integração possibilita uma experiência mais fluida e conveniente, conectando o processo de escolha do filme à etapa final de compra dos ingressos, sem que os usuários precisem buscar manualmente as sessões disponíveis.

Benefícios da Integração: A transição entre a votação no aplicativo e a busca pelo filme no cinema ocorre de maneira automática, eliminando a necessidade de pesquisas manuais. A funcionalidade garante recomendações mais relevantes e acessíveis. Além disso, o uso da API da **Ingresso.com** otimiza o tempo necessário para encontrar informações sobre sessões e cinemas.

A integração com a API da Ingresso.com conecta de forma prática o momento da decisão com a ação de ir ao cinema, reforçando a proposta do **Match Movie** de simplificar escolhas em grupo e proporcionar uma experiência completa e conectada.

Capítulo 5

Avaliação e Resultados

Este capítulo apresenta a análise dos resultados obtidos a partir da aplicação de questionários de usabilidade e aceitação do aplicativo **Match Movie** para 5 participantes, que serão chamados nas Tabelas de *p1*, *p2*, *p3*, *p4* e *p5*. O objetivo dessas avaliações foi compreender como os usuários percebem a interface, a experiência de uso e o impacto do aplicativo na escolha de filmes em grupo.

Os participantes responderam às perguntas utilizando uma escala *Likert* [?] de 1 a 5, onde:

- 1 - Discordo totalmente
- 2 - Discordo
- 3 - Neutro
- 4 - Concordo
- 5 - Concordo totalmente

5.1 Avaliação do TAM e Usabilidade

Para validar a aceitação e usabilidade do **Match Movie**, foi elaborado um questionário baseado no **Modelo de Aceitação de Tecnologia** (*Technology Acceptance Model: Modelo de Aceitação de Tecnologia (TAM)*) e na Escala de Usabilidade do Sistema (*System Usability Scale - SUS*), conforme a adaptação proposta por ?. O questionário foi distribuído para cinco colaboradores e estruturado em torno de três fatores principais:

- ***Perceived Ease of Use: Facilidade de Uso Percebida (PEOU)***: grau em que o usuário percebe a tecnologia como fácil de aprender e utilizar.
- ***Perceived Usefulness: Utilidade Percebida (PU)***: percepção de utilidade da tecnologia na melhoria da produtividade ou na facilitação de uma tarefa.

- **Intention to Use: Intenção de Uso (IU):** probabilidade de o usuário continuar utilizando o aplicativo e recomendá-lo a outras pessoas.

Os resultados coletados foram organizados e analisados para cada uma das dimensões do TAM.

5.1.1 Facilidade de Uso Percebida (PEOU)

A **Facilidade de Uso Percebida (PEOU)** mede o quão intuitivo e acessível o aplicativo é para os usuários. Esse critério é essencial para garantir que o **Match Movie** possa ser utilizado sem necessidade de um aprendizado extenso. Tabela 7.

Pergunta	Respostas (Distribuição em %)
O aplicativo é fácil de usar?	- 60% concordam totalmente - 40% concordam - 0% neutros - 0% discordam - 0% discordam totalmente
As funcionalidades são intuitivas e de fácil acesso?	- 60% concordam totalmente - 40% concordam - 0% neutros - 0% discordam - 0% discordam totalmente
Conseguir aprender a usar o aplicativo rapidamente?	- 40% concordam totalmente - 60% concordam - 0% neutros - 0% discordam - 0% discordam totalmente
A interface mantém uma consistência visual em todas as telas?	- 60% concordam totalmente - 20% concordam - 20% neutros - 0% discordam - 0% discordam totalmente
Os ícones e botões são padronizados e compreensíveis?	- 60% concordam totalmente - 20% concordam - 20% neutros - 0% discordam - 0% discordam totalmente

Tabela 7: Resultados da pesquisa sobre Facilidade de Uso Percebida (PEOU)

Os resultados indicam que a maioria dos participantes considerou a interface intuitiva e de fácil aprendizado. Elementos como a organização das telas e a clareza das funcionalidades foram destacados como pontos positivos.

5.1.2 Utilidade Percebida (PU)

A **Utilidade Percebida (PU)** avalia se os usuários enxergam valor no aplicativo para otimizar a escolha de filmes em grupo, verificando se ele realmente facilita o processo. Tabela 8.

Pergunta	Respostas (Distribuição em %)
Conseguir escolher um filme com o grupo ficou mais rápido?	- 20% concordam totalmente - 80% concordam - 0% neutros - 0% discordam - 0% discordam totalmente
As recomendações de filmes são relevantes?	- 60% concordam totalmente - 40% concordam - 10% neutros - 0% discordam - 0% discordam totalmente
A integração com serviços externos facilitou a escolha de um filme?	- 60% concordam totalmente - 40% concordam - 0% neutros - 0% discordam - 0% discordam totalmente
O aplicativo realmente ajudou a reduzir o tempo para decidir qual filme assistir?	- 40% concordam totalmente - 60% concordam - 0% neutros - 0% discordam - 0% discordam totalmente
O aplicativo oferece sugestões que combinam com meus interesses e gostos?	- 60% concordam totalmente - 40% concordam - 0% neutros - 0% discordam - 0% discordam totalmente

Tabela 8: Resultados da pesquisa sobre Utilidade Percebida (PU)

Os participantes relataram que o aplicativo cumpre sua proposta, oferecendo uma maneira prática de selecionar filmes com base em preferências coletivas. Além disso, a integração com serviços externos, como **TheMovieDB** e **Ingresso.com**, foi considerada um diferencial positivo.

5.1.3 Intenção de Uso (IU)

A **Intenção de Uso (IU)** analisa se os usuários pretendem continuar utilizando o aplicativo e recomendá-lo para outras pessoas. Tabela 9.

Pergunta	Respostas (Distribuição em %)
Eu utilizaria o aplicativo novamente para decidir filmes com amigos?	- 40% concordam totalmente - 60% concordam - 0% neutros - 0% discordam - 0% discordam totalmente
O aplicativo melhorou minha experiência na escolha de filmes?	- 60% concordam totalmente - 40% concordam - 0% neutros - 0% discordam - 0% discordam totalmente
Recomendaria o aplicativo para outras pessoas?	- 60% concordam totalmente - 40% concordam - 0% neutros - 0% discordam - 0% discordam totalmente
Pretendo continuar usando o aplicativo regularmente?	- 60% concordam totalmente - 40% concordam - 0% neutros - 0% discordam - 0% discordam totalmente

Tabela 9: Resultados da pesquisa sobre Intenção de Uso (IU)

Os dados mostram que o **Match Movie** tem potencial de aceitação, com uma percepção positiva entre os usuários. A tendência de recomendação e o uso contínuo indicam que o aplicativo atende bem às expectativas. No entanto, novas iterações podem ser exploradas para aprimorar ainda mais a experiência do usuário.

5.2 Avaliação de UI/UX por meio de formulário

Além da aceitação tecnológica, foi aplicada uma avaliação voltada à **experiência do usuário (UI/UX)**, considerando aspectos como design, acessibilidade, eficiência e feedback visual. Para isso, os participantes responderam a um questionário específico utilizando a mesma escala *Likert* mencionada anteriormente.

5.2.1 Facilidade de Uso

A facilidade de uso foi um dos pontos centrais da avaliação, buscando compreender se os usuários consideram o Match Movie intuitivo e de fácil aprendizado. Os resultados mostraram que, respectivamente, 40% e 60% dos participantes concordam e concordam totalmente que a

navegação no aplicativo é simples e que as funcionalidades são acessíveis sem a necessidade de instruções complexas. Tabela 10.

Pergunta	Respostas (Distribuição em %)
O aplicativo é fácil de usar?	- 40% concordam totalmente - 60% concordam - 0% neutros - 0% discordam - 0% discordam totalmente
As funcionalidades são intuitivas e de fácil acesso?	- 20% concordam totalmente - 80% concordam - 0% neutros - 0% discordam - 0% discordam totalmente
Consegui aprender a usar o aplicativo rapidamente?	- 40% concordam totalmente - 40% concordam - 20% neutros - 0% discordam - 0% discordam totalmente

Tabela 10: Resultados da pesquisa sobre Facilidade de Uso

Isso indica que o design e a estrutura do aplicativo foram bem planejados, garantindo que novos usuários possam utilizá-lo sem dificuldades.

5.2.2 Design da Interface

O design da interface avalia a organização visual do aplicativo, incluindo o uso de cores, tipografia e distribuição dos elementos na tela. Segundo os dados coletados, 40% concordam totalmente, 40% concordam e 20% são neutros em relação a considerarem o design bem estruturado e agradável. Tabela 11.

Pergunta	Respostas (Distribuição em %)
A interface é visualmente agradável?	- 40% concordam totalmente - 40% concordam - 20% neutros - 0% discordam - 0% discordam totalmente
O design é limpo e sem elementos desnecessários?	- 60% concordam totalmente - 40% concordam - 0% neutros - 0% discordam - 0% discordam totalmente

Tabela 11: Resultados da pesquisa sobre Design da Interface

Esse resultado reforça que a escolha do estilo visual foi adequada para proporcionar uma experiência fluida, sem sobrecarregar o usuário com informações desnecessárias.

5.2.3 Experiência do Usuário

A experiência do usuário (**UX**) foi analisada a partir de percepções sobre a jornada dentro do aplicativo, desde o primeiro acesso até a realização de uma escolha de filme. Os dados indicam que 100% dos participantes tiveram uma experiência bastante positiva, destacando a clareza das etapas e a interatividade do sistema. Tabela 12.

Pergunta	Respostas (Distribuição em %)
Estou satisfeito com a experiência proporcionada pelo aplicativo?	- 80% concordam totalmente - 20% concordam - 0% neutros - 0% discordam - 0% discordam totalmente
O aplicativo atendeu às minhas expectativas iniciais?	- 100% concordam totalmente - 0% concordam - 0% neutros - 0% discordam - 0% discordam totalmente

Tabela 12: Resultados da pesquisa sobre Experiência do Usuário

Isso sugere que o **Match Movie** foi bem-sucedido em criar um fluxo de uso coerente e satisfatório para seus usuários.

5.2.4 Eficiência

A eficiência do aplicativo mede o quão rápido e direto o usuário consegue atingir seu objetivo – neste caso, escolher um filme para assistir com um grupo. Os resultados apontam, novamente, que 100% dos participantes sentiram que o aplicativo facilitou a tomada de decisão de forma ágil e organizada. Tabela 13.

Pergunta	Respostas (Distribuição em %)
Consigo realizar minhas tarefas de forma rápida e eficiente?	- 0% concordam totalmente - 100% concordam - 0% neutros - 0% discordam - 0% discordam totalmente
O aplicativo responde de maneira ágil às minhas interações?	- 60% concordam totalmente - 40% concordam - 0% neutros - 0% discordam - 0% discordam totalmente

Tabela 13: Resultados da pesquisa sobre Eficiência

Com isso, fica evidente que a aplicação atende às expectativas de rapidez e praticidade, sem gerar frustrações no processo de seleção.

5.2.5 Acessibilidade

A acessibilidade foi outro ponto de avaliação, levando em conta aspectos como contraste de cores, tamanho das fontes e adaptabilidade para diferentes públicos. Segundo as respostas coletadas, a grande maioria dos usuários consideraram os textos legíveis e os elementos visuais bem distribuídos. Tabela 14.

Pergunta	Respostas (Distribuição em %)
As fontes e textos são de fácil leitura?	- 80% concordam totalmente - 20% concordam - 0% neutros - 0% discordam - 0% discordam totalmente
A estrutura de navegação é clara e lógica?	- 60% concordam totalmente - 40% concordam - 0% neutros - 0% discordam - 0% discordam totalmente

Tabela 14: Resultados da pesquisa sobre Acessibilidade

Esse dado é fundamental para garantir que o aplicativo possa ser utilizado por pessoas com diferentes necessidades, reforçando um design inclusivo.

5.2.6 Feedback do usuário

O feedback do usuário é essencial para aprimorar continuamente a aplicação. Os participantes foram questionados sobre a clareza das mensagens de erro, confirmações de ação e orientações dentro do sistema. Os dados mostram que 100% dos usuários avaliaram positivamente esse aspecto. Tabela 15.

Pergunta	Respostas (Distribuição em %)
O aplicativo informa claramente o que está acontecendo durante o uso?	- 80% concordam totalmente - 20% concordam - 0% neutros - 0% discordam - 0% discordam totalmente
Recebo feedback imediato após realizar ações?	- 20% concordam totalmente - 80% concordam - 0% neutros - 0% discordam - 0% discordam totalmente

Tabela 15: Resultados da pesquisa sobre Feedback do Usuário

Isso indica que o aplicativo fornece informações úteis durante o uso, auxiliando o usuário a entender suas opções e a navegar com confiança.

Os resultados gerais desta avaliação demonstram que a interface do **Match Movie** foi bem recebida, atendendo aos critérios essenciais de UI/UX. Embora os índices sejam positivos, pequenos ajustes poderão ser feitos para aprimorar ainda mais a experiência, garantindo um uso ainda mais intuitivo e agradável. Dessa forma, o aplicativo continuará evoluindo conforme as necessidades dos usuários.

Capítulo 6

Considerações Finais e Trabalhos Futuros

O desenvolvimento do Match Movie demonstrou como a integração de tecnologias modernas pode transformar a experiência de escolha e organização de atividades em grupo. Ao combinar funcionalidades como votação colaborativa e integração com APIs, como a da Ingresso.com, o aplicativo oferece uma solução prática e eficiente, facilitando a tomada de decisões e o acesso a informações relevantes, como horários e cinemas. Essa abordagem reforça a importância de alinhar tecnologia e usabilidade para atender às necessidades dos usuários de maneira intuitiva e acessível, consolidando o Match Movie como uma ferramenta inovadora para otimizar interações sociais e escolhas coletivas.

Para trabalhos futuros, há oportunidades de expandir o Match Movie com novas funcionalidades que aprimorem ainda mais a experiência do usuário. Dentre as possibilidades, destaca-se a implementação de algoritmos de recomendação mais avançados, baseados em aprendizado de máquina, para personalizar as sugestões de filmes com maior precisão. Além disso, a integração com outras APIs, como plataformas de streaming e eventos culturais, pode ampliar o escopo do aplicativo, permitindo que os usuários explorem atividades diversas além do cinema. Por fim, a melhoria da acessibilidade do aplicativo, com foco em atender às diretrizes internacionais como o WCAG, pode tornar o Match Movie ainda mais inclusivo, abrangendo uma maior diversidade de públicos e consolidando sua proposta como uma solução completa e conectada.

O código-fonte do Match Movie está disponível publicamente, permitindo que a comunidade acompanhe e contribua para seu desenvolvimento. O repositório do front-end pode ser acessado em MatchMovie-Native, enquanto o repositório da API está disponível em MatchMovie-API. Esses repositórios possibilitam não apenas a continuidade do projeto, mas também a exploração de novas melhorias e integrações que possam ampliar sua usabilidade e impacto.

Apêndice

A Diagrama de Classe

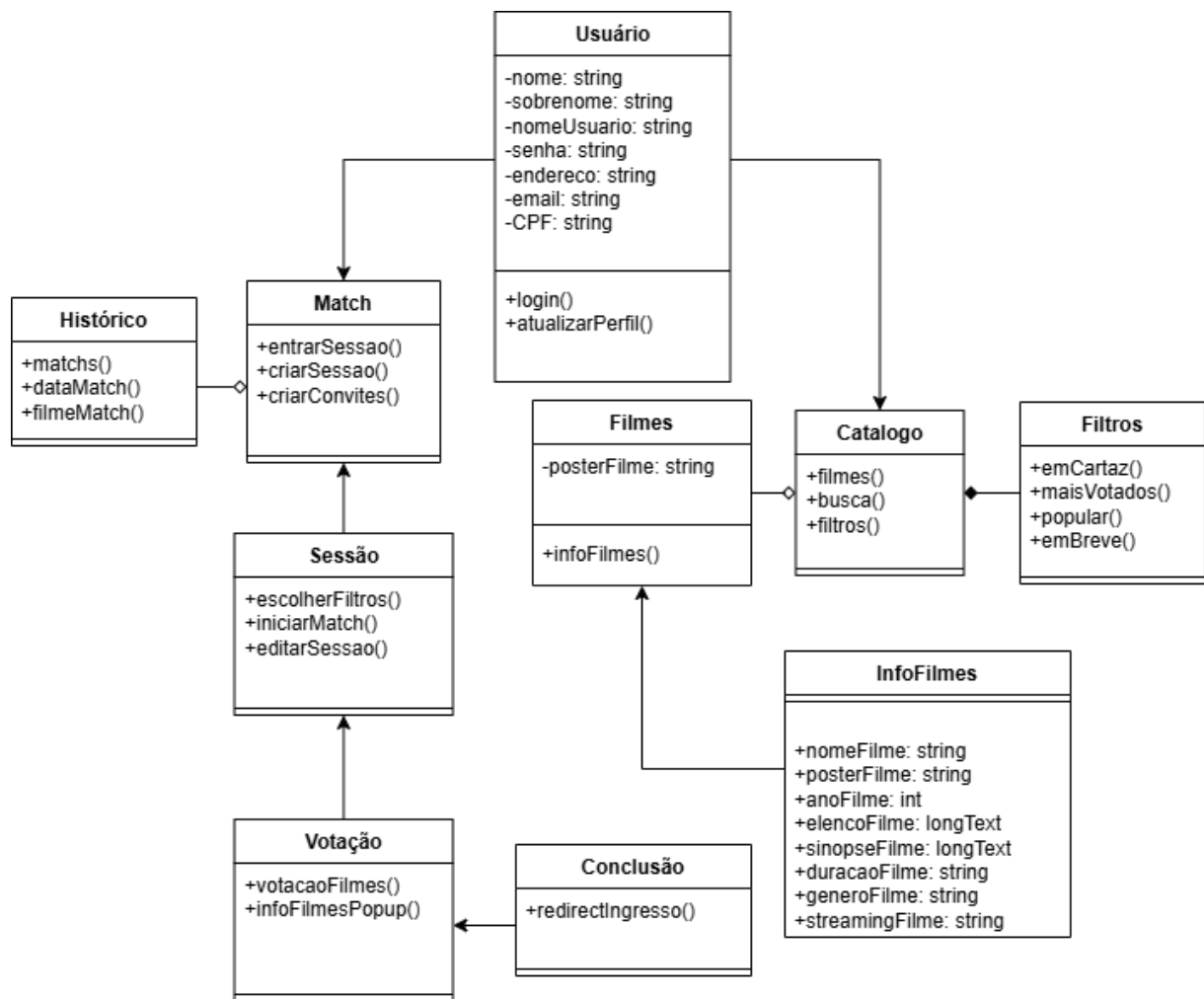


Figura 19: Diagrama de classe do Match Movie. Elaborado pelo autor.

C Diagrama de Sequência - Login de usuário

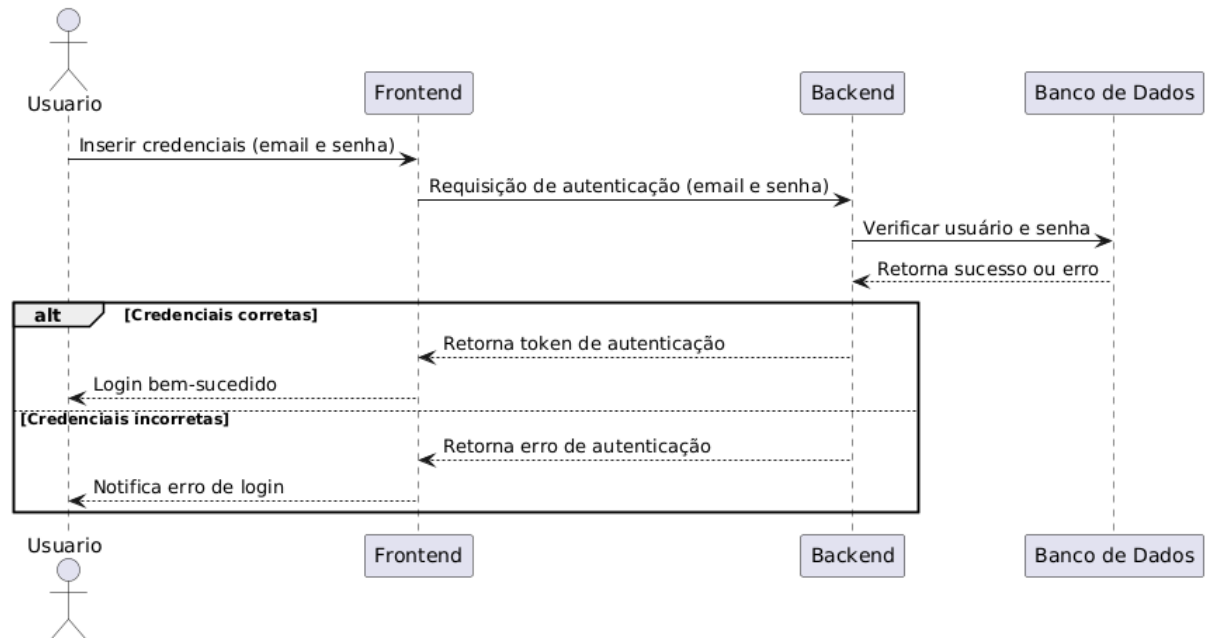


Figura 21: Diagrama de Sequência - Login de usuário. Elaborado pelo autor.

D Diagrama de Sequência - Registro de usuário

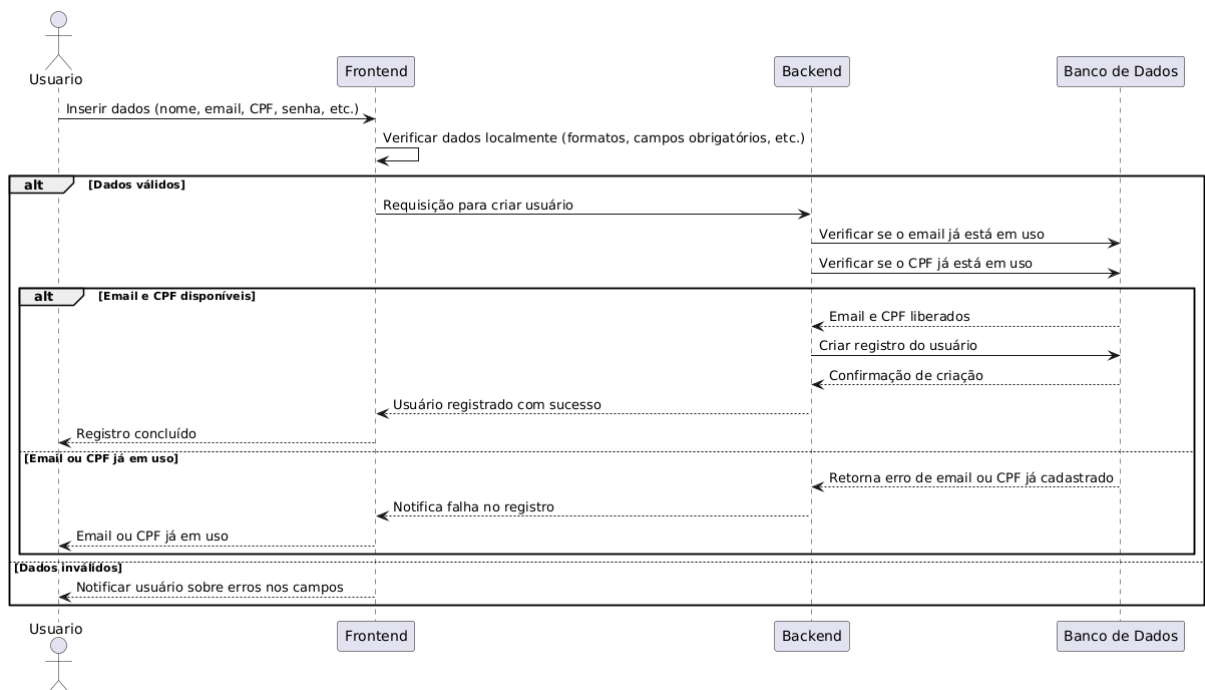


Figura 22: Diagrama de Sequência - Registro de usuário. Elaborado pelo autor.

E Diagrama de Sequência - Obtenção de filmes

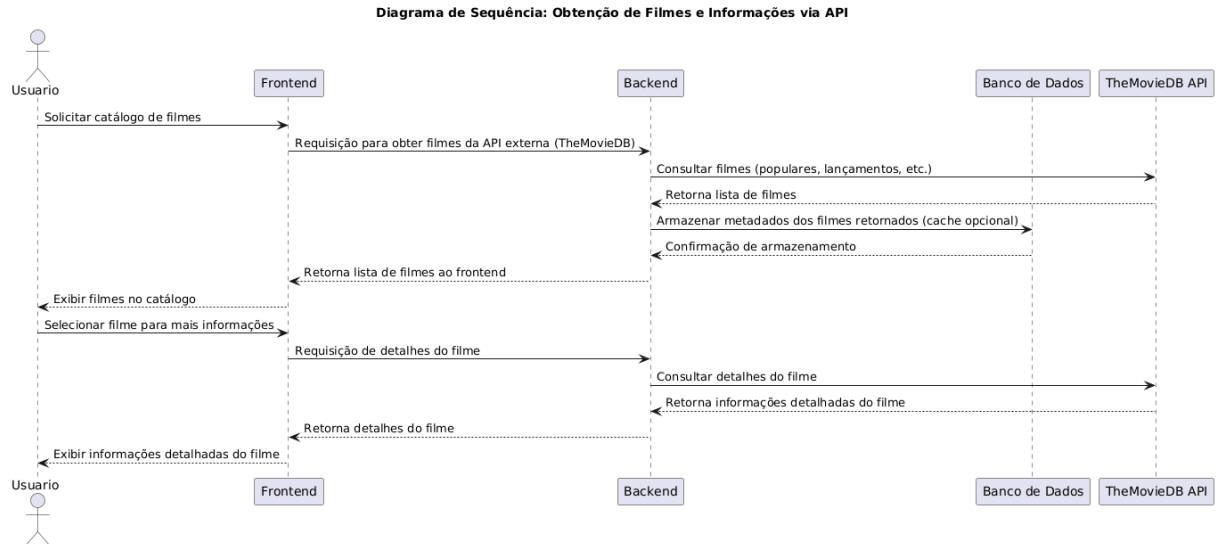


Figura 23: Diagrama de Sequência - Obtenção de filmes. Elaborado pelo autor.

F Diagrama de Sequência - Marcar filmes como favorito ou assistido

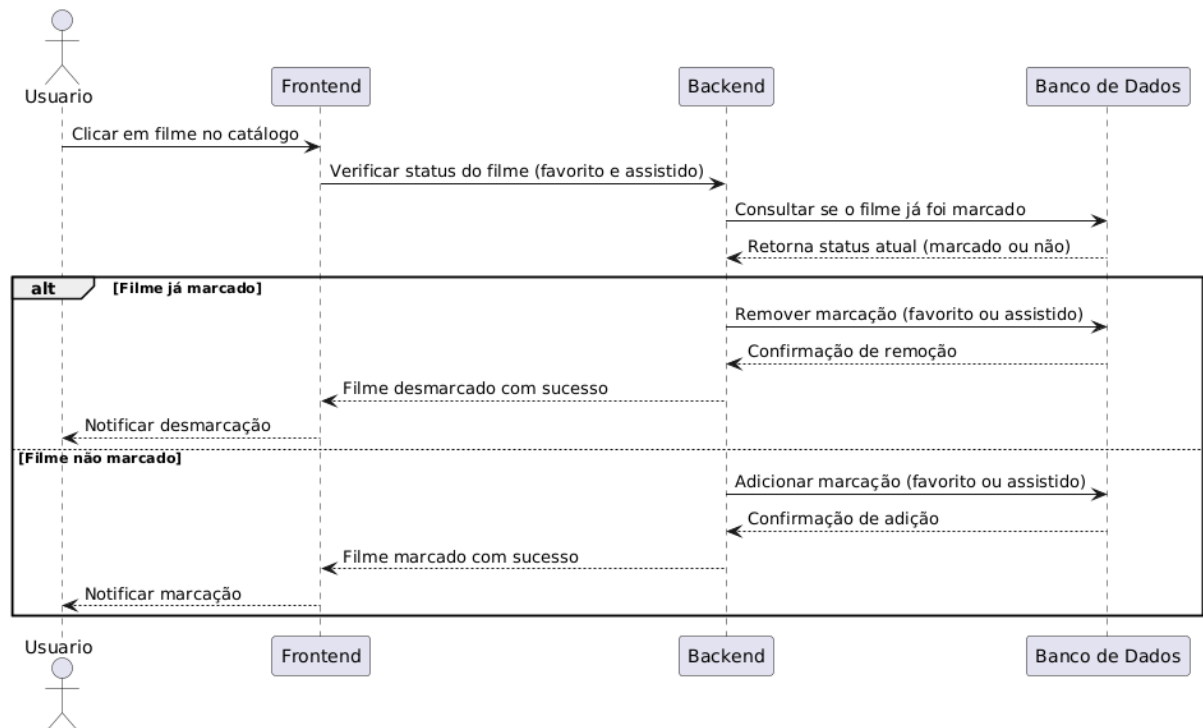


Figura 24: Diagrama de Sequência - Marcar filmes como favorito ou assistido. Elaborado pelo autor.

G Diagrama de Sequência - Votação de filmes (match)

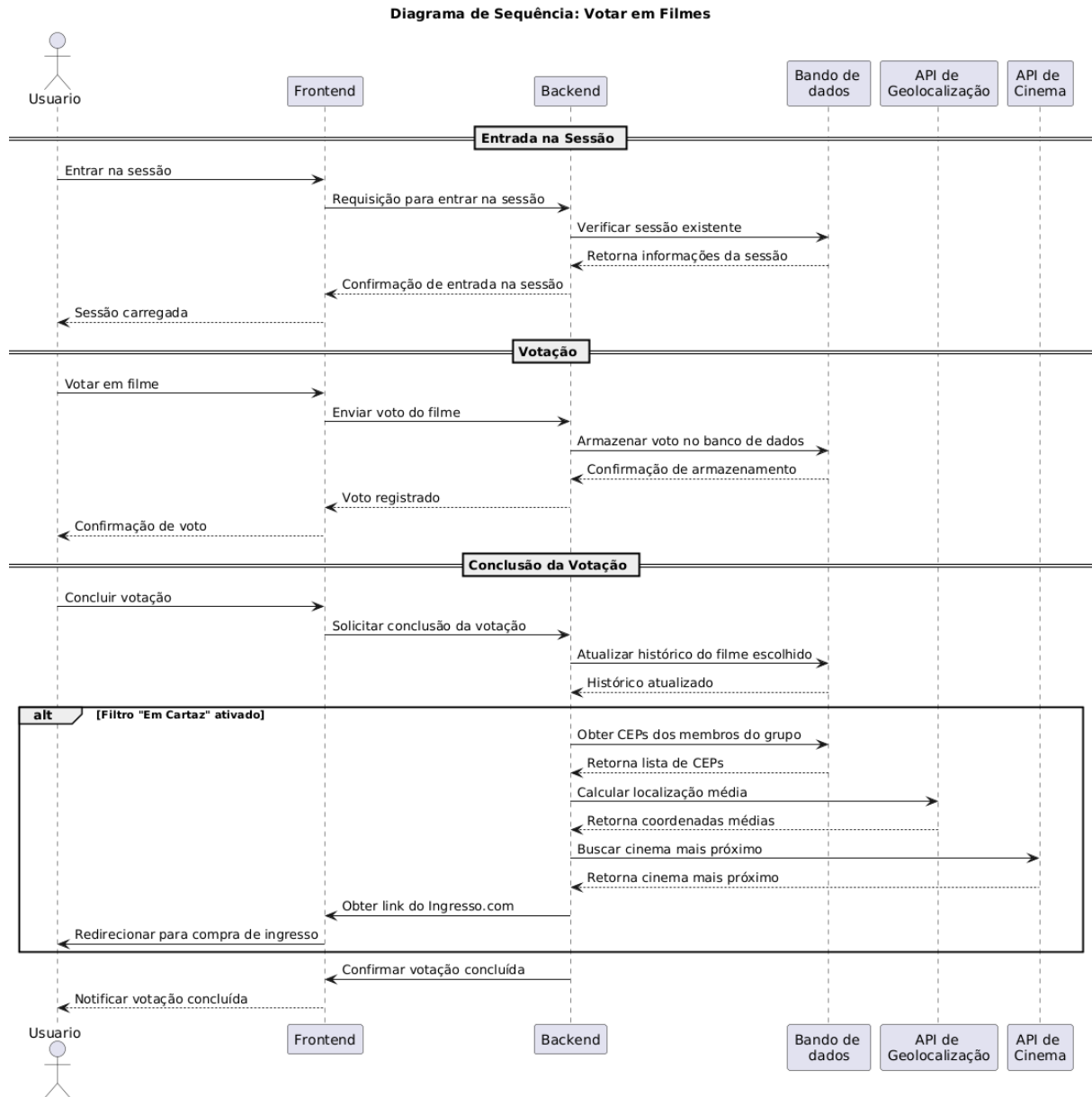


Figura 25: Diagrama de Sequência - Votação de filmes (match). Elaborado pelo autor.

H Arquitetura do sistema modelo C4

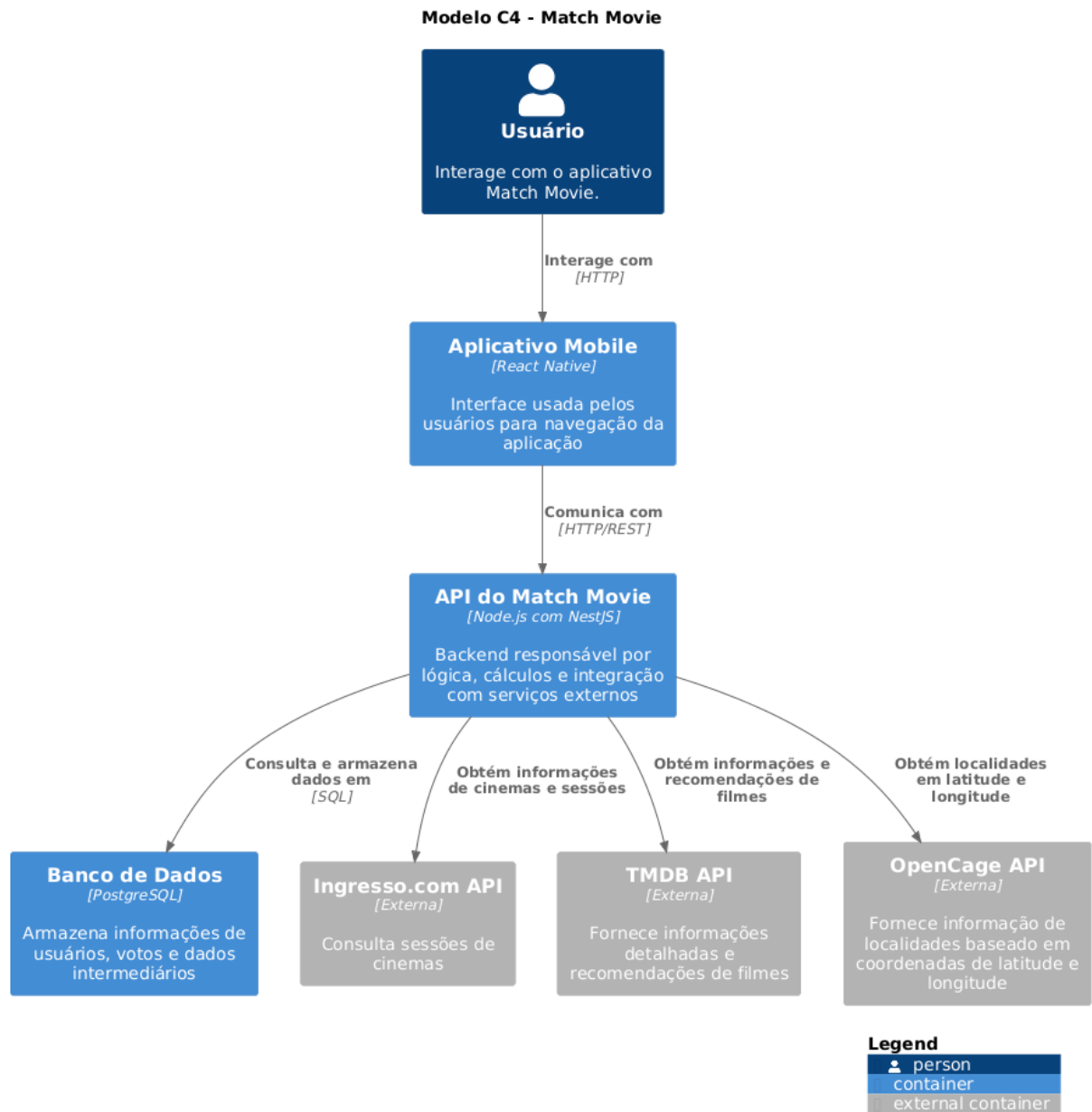


Figura 26: Figura de exemplo da arquitetura do sistema no modelo C4. Elaborado pelo autor.

I Swagger - Endpoints de autenticação

auth			^
POST	/auth/login	Fazer login	▼
POST	/auth/request-password-reset	Solicitar redefinição de senha	▼
POST	/auth/reset-password	Resetar senha	▼

Figura 27: Figura de exemplo dos endpoints de autenticação no Swagger. Elaborado pelo autor.

J Swagger - Endpoints de usuários

users			^
POST	/users	Criar usuário	▼
GET	/users	Listar usuários	🔒 ▼
GET	/users/{id}	Listar usuário	🔒 ▼
PATCH	/users/{id}	Atualizar usuário	🔒 ▼
DELETE	/users/{id}	Excluir usuário	🔒 ▼
GET	/users/username/{username}	Procurar usuário por username	🔒 ▼
POST	/users/{id}/upload-profile-picture	Adicionar foto de perfil para usuário	🔒 ▼
GET	/users/{userId}/groups	Listar os grupos do usuário	🔒 ▼

Figura 28: Figura de exemplo dos endpoints de CRUD de usuários. Elaborado pelo autor.

K Swagger - Endpoints de grupos

groups			^
POST	/groups	Criar um grupo	🔒 ▼
GET	/groups	Listar grupos	🔒 ▼
GET	/groups/{id}	Listar um grupo	🔒 ▼
PATCH	/groups/{id}	Atualizar um grupo	🔒 ▼
DELETE	/groups/{id}	Deletar um grupo	🔒 ▼
POST	/groups/{groupId}/users/{userId}	Adicione usuário no grupo	🔒 ▼
DELETE	/groups/{groupId}/users/{userId}	Remova usuário do grupo	🔒 ▼
GET	/groups/{groupId}/users	Liste os usuários do grupo	🔒 ▼

Figura 29: Figura de exemplo dos endpoints de CRUD de grupos. Elaborado pelo autor.

L Swagger - Endpoints de filmes

movies			^
GET	/movies/popular	Listar filmes populares	▼
GET	/movies/topRated	Listar filmes melhor avaliados	▼
GET	/movies/now_playing	Listar filmes em cartaz	▼
GET	/movies/upcoming	Listar filmes em breve	▼
GET	/movies/search	Listar filmes por nome	▼
GET	/movies/{movieId}/details	Listar detalhes do filme	▼
GET	/movies/{movieId}/watch_providers	Listar streaming disponíveis do filme	▼
GET	/movies/{moviePoster}/poster	Listar poster do filme	▼

Figura 30: Figura de exemplo dos endpoints de filmes. Elaborado pelo autor.

M Swagger - Endpoints de favoritos

favorites			^
GET	/favorites/isFavorite/{userId}/{movieId}	Verificar se filme já foi marcado como favorito ou não	🔒 ▼
POST	/favorites	Marcar filme como favorito	🔒 ▼
GET	/favorites/count/{userId}	Contar quantidade de favoritos do usuário	🔒 ▼
GET	/favorites/{userId}	Listar filmes marcados como favorito	🔒 ▼

Figura 31: Figura de exemplo dos endpoints de filmes favoritos do usuário. Elaborado pelo autor.

N Swagger - Endpoints de assistidos

watched			^
GET	/watched/isWatched/{userId}/{movieId}	Verificar se filme já foi marcado como assistido ou não	🔒 ▼
POST	/watched	Marcar filme como assistido	🔒 ▼
GET	/watched/count/{userId}	Contar quantidade de assistidos do usuário	🔒 ▼
GET	/watched/{userId}	Listar filmes marcados como assistido	🔒 ▼

Figura 32: Figura de exemplo dos endpoints de filmes assistidos do usuário. Elaborado pelo autor.

O Swagger - Endpoints de geolocalização e ingresso.com

geolocation		^
GET	/geolocation/{groupId}/midpoint	Calcular ponto médio de latitude entre usuários
ingresso		^
GET	/ingresso/city/{cityId}	Pegar a url do cinema mais próximo para redirecionamento.
GET	/ingresso/lat/{lat}/lng/{lng}	Listar o cinema mais próximo baseado na latitude e longitude média dos usuários do grupo

Figura 33: Figura de exemplo dos endpoints de geolocalização e do ingresso.com. Elaborado pelo autor.

P Swagger - Endpoints de match (votação)

match		^
POST	/match/{groupId}/start/{movieId}	Começar match de grupo
POST	/match/{groupId}/vote	Receber votos de grupo
GET	/match/{groupId}/recommendations	Listar filmes recomendados baseado em filme do grupo
POST	/match/{groupId}/check	Checar match do grupo
GET	/match/{groupId}/history	Checar historico de match do grupo
DELETE	/match/{id}	Deletar match do grupo

Figura 34: Figura de exemplo dos endpoints votação de filmes. Elaborado pelo autor.

Referências